Democratic and Popular Republic of Algeria
Ministry of Higher Education and Scientific Research
Higher School of Computer and Digital Sciences and Technologies
MI department

2CP Project

" MULTIDISCIPLINARY PROJECT "

Theme

IMPLEMENTATION AND DESIGN OF A MOBILE LEARNING APPLICATION:

SKILLZONE

Performed by:
BENHAOUA Boualem
BENZEKRI Brahim
KOUDA Rania
MEFTAH Wassim Abdallah
MOUAICI Agnès

Supervised by:
Mlle. ALLOU Lydia Imene

Academic Year: 2024/2025

Contents

List of Figures

List of Tables

Abbreviation List

Abbreviation	Meaning
API	Application Programming Interface
CMS	Content Management System
CRUD	Create, Read, Update, Delete (referring to basic operations in database management)
CSS	Cascading Style Sheets
DBMS	Database Management System
GUI	Graphical User Interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
HTTPS	Hypertext Transfer Protocol Secure
JS	JavaScript
LaTeX	Lamport TeX
MI	Management and Informatics
React JS	React JavaScript library
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Acknowledgements

"Learn, grow, and shine with SkillZone !"

We would like to express our deep gratitude to everyone who contributed to the realization of the **Skillzone** project.

First, we warmly thank our supervisor, **Mlle Allou Lydia Imene**, for her availability, kind support, and insightful guidance throughout this journey. Her mentorship was a key pillar in structuring our work and bringing our vision to life.

We also wish to express our appreciation to our entire team for their dedication, discipline, and collaborative spirit. Each member brought their unique value to the project, allowing us to design an innovative mobile application focused on personal and professional development through a gamified learning system—especially Benzekri Brahim and Meftah Wassim Abdallah, whose contributions were invaluable.

We are equally grateful to the users who tested the application for their constructive feedback and enthusiasm. Their input greatly helped us improve the user experience and better respond to the real needs of our target audience. Finally, we sincerely thank our families and friends for their constant moral support, patience, and encouragement throughout this adventure.

We are confident that **SkillZone** offers a useful and accessible solution to help individuals grow their skills and make a positive impact on their personal and professional journeys.

Skillzone Team

General Introduction

In a world where continuous learning has become essential, digital platforms play a major role in transforming how individuals acquire new skills. Globally, access to online education has expanded significantly; however, in Algeria, opportunities for flexible and accessible learning remain limited. Many people still face financial and accessibility barriers that hinder their personal and professional development.

At the same time, technological innovations in education are rapidly advancing. From mobile applications to gamified learning experiences, technology is revolutionizing the way knowledge is delivered and absorbed. Learners are increasingly seeking platforms that offer flexibility, engagement, and accessibility, while educators are exploring new methods to deliver content. However, a major challenge remains: ensuring that digital education is accessible to all social categories, particularly those with limited resources.

In response to this need, we propose the development and implementation of SkillZone, a mobile application designed to democratize access to skill-building in Algeria. SkillZone introduces an innovative points-based system, allowing users to unlock educational content with or without financial payment. By combining training in both technical (hard skills) and interpersonal (soft skills) competencies, SkillZone aims to empower students, developers, and all individuals eager to learn—regardless of their financial situation.

Our approach is structured into three distinct chapters::

The first chapter, **Preliminary Study and Needs Analysis**, offers a comprehensive examination of existing educational platforms, evaluates their features, and identifies areas for improvement.

The second chapter, **Design and modeling of system components**, outlines the functional and technical specifications required for application development, identifying key stakeholders, and providing use case diagrams, sequence diagrams, class diagrams, and user interface design.

Finally, the third chapter, **Implementation**, ddetails the development process, the tools used, the deployment environment, and showcases the main features of the platform along with usage guidelines.

We will conclude this report with a summary of our findings, highlighting key lessons learned and proposing directions for the future development of SkillZone.

Chapter 1

Preliminary Study and Needs Analysis

1.1 Introduction

Before we build any project, we need to understand the market. It's not enough to just have a general idea, we need real information. That means looking at what people need, what exists already, and what works. In this first chapter, we will explore the current market for online learning platforms that focus on skills. We will look at international platforms and learn from their experience. Our goal is to understand what works, what doesn't, and where SkillZone fits in.

1.2 Project Presentation

SkillZone s a platform that helps people develop two types of skills:

- **Hard skills**, such as programming, using digital tools, or UI/UX design.
- **Soft skills**, such as public speaking, leadership, time management, or teamwork.

Many young people know they need to learn these skills, but they don't know how or where to start. SkillZone offers structured learning paths, hands-on projects, progress tracking, and a learning-motivating space.

Our goal is to create a simple and accessible platform, specially designed for youth in Algeria.

1.3 Team Members Contributions

1.3.1 Meftah Wassim Abdallah (Team Leader)

- Developed most of the backend APIs (**Skillzone Backend Repository**), including:
  - **Users**: register, verify-email, login, logout, profile, update-profile, change-password, update-points, update-refresh-token.
  - **Quizzes**: quiz-list, quiz-detail, start-quiz, submit-quiz, quiz-statistics.
  - **Courses**: course-list, course-detail, course-statistics, unlock-course , upload-course .
  - **Achievements**: all core functionality.
- Deployed the backend APIs on Render (*Visit Skillzone API Site*).

- Set up the PostgreSQL database on Render.
- Created the class diagram with explanations.
- Designed UI pages for:
  - Email verification
  - Notifications
  - Quizzes (Game, Results)
- Designed the 4 onboarding/intro UI screens.
- Enhanced the mobile UI color palette and redesigned the logo.
- Wrote the second technical report.

1.3.2 Kouda Rania

- Wrote the first technical report.
- Authored most of the final report project :
  - the general introduction with chapter 1 (Preliminary Study).
  - Chapter 3 (Implementation), and the general conclusion.
  - the webography and the bibliography.
- Designed the UI of all of the website pages.
- Developed and linked the entire website front-end (check the GitHub project description file(`README.md`)).
  - The landing page.
  - About us page.
  - Terms of service page.
- Deployed the website on Netlify.
- Created the use case diagram (for both teacher and student) with explenation.

1.3.3 Benzekri Brahim

- Designed UI pages for:
  - Course details.
  - Course upload form.
  - Avatar editing.
  - Inventory.
  - Points and achievements.
  - Lesson video player.
- Developed most of the mobile front-end (19/24 screens), (check the GitHub project description file (`README.md`)).

- Integrated the backend APIs with the mobile application.
- Implemented additional features:
  - Video player for lessons.
  - Video picking from gallery.
  - Local storage of avatar and authentication tokens.
  - Centralized APIs and Storage system for smooth user experience.
- Linked the website to the mobile application.

1.3.4 Mouaici Agnès

- Proposed the initial idea of the project.
- Created the first design version of all mobile app pages:
  - Logo, UI colors, login and registration, Quizzes, home qnd search pages.
  - Current points, paiment card, profile and Teacher page.
- Designed the initial version of the website landing page.
- Developed backend courses APIs.
- Designed the global architecture diagram with explanation.

1.3.5 Benhaoua Boualem

- Created the project presentation slides.
- Developed the first 5 pages of the mobile front-end pages.
- Built initial Users auth API.
- Created sequence diagrams with explanations.

1.4 Problematic

A lot of students and young people know that they need more skills to succeed in both their personal and professional life, but they don't know where to start. Some try free content online, but it's often not enough. Others get overwhelmed or lose motivation. On top of that, schools don't always teach practical skills like coding or public speaking. This creates a gap between what young people know and what the job market expects. Our question is: how can we use a modern platform to help people learn real skills, in a simple and motivating way? We also need to look at what's already been done—what works, what fails—and create something better for our local context.

1.5 Study of the Similar Educational Platforms

We have selected the following educational platforms for analysis: Udemy, Coursera and OpenClassrooms. Their details are presented below:

Udemy

Udemy is a global learning platform offering a wide variety of courses across multiple disciplines for learners of all levels.

- Website : `https://www.udemy.com/`
- Technologies used :Python, Django, JavaScript
- Year of Creation: 2010
- Number of Courses: Over 210,000 courses
- Number of Users: Over 60 million students worldwide
- Mobile Application: Available with over 10 million downloads
- Features for Users:
  - On-demand video courses.
  - Certificates of completion.
  - Lifetime access to purchased courses.
  - Multilingual content.
  - Frequent sales and promotions.

The figure 1.1 represent main functions in landing page of Udemy web application:

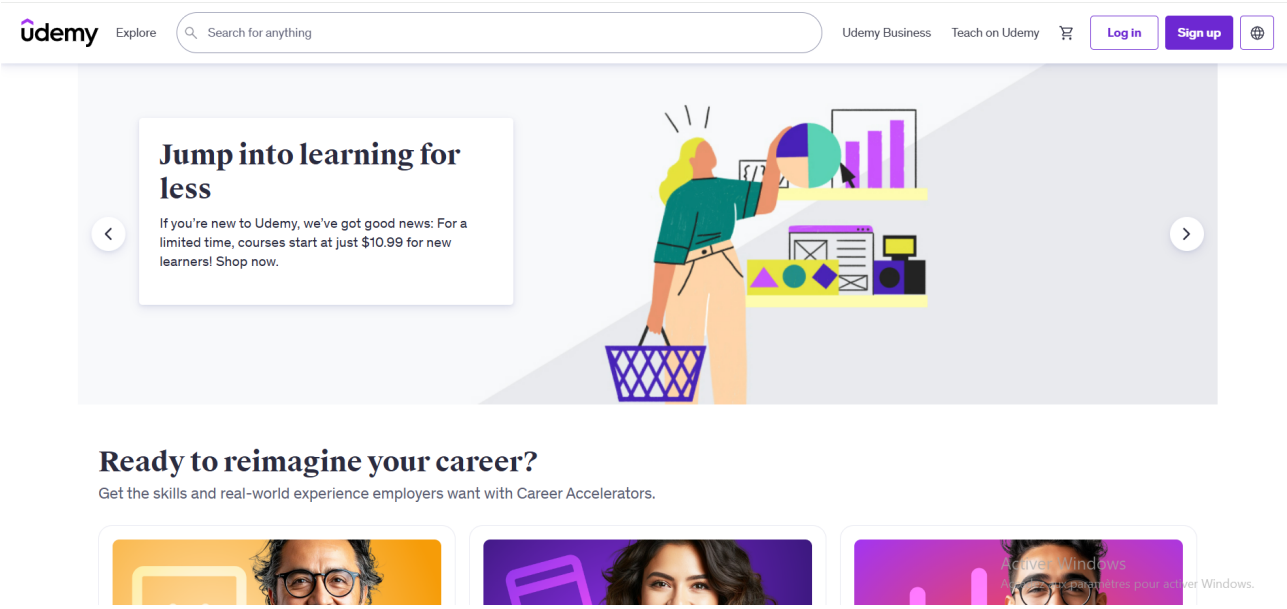


Figure 1.1: Landing page of Udemy

Coursera

Coursera partners with top universities and organizations all around the world to offer online courses, certifications, and degrees.

- Website : `https://www.coursera.org/`
- Technologies used :Java, React, Python
- Year of Creation: 2012
- Number of Courses: Over 7,000 courses
- Number of Users: Over 120 million learners
- Mobile Application: Available with over 5 million downloads
- Features for Users:
  - University-accredited certificates.
  - Guided projects and specializations.
  - Self-paced and instructor-led options.
  - Financial aid and free auditing options.
  - Peer-reviewed assignments.

The figure 1.3 represent the interface of Coursera web application for individuals :

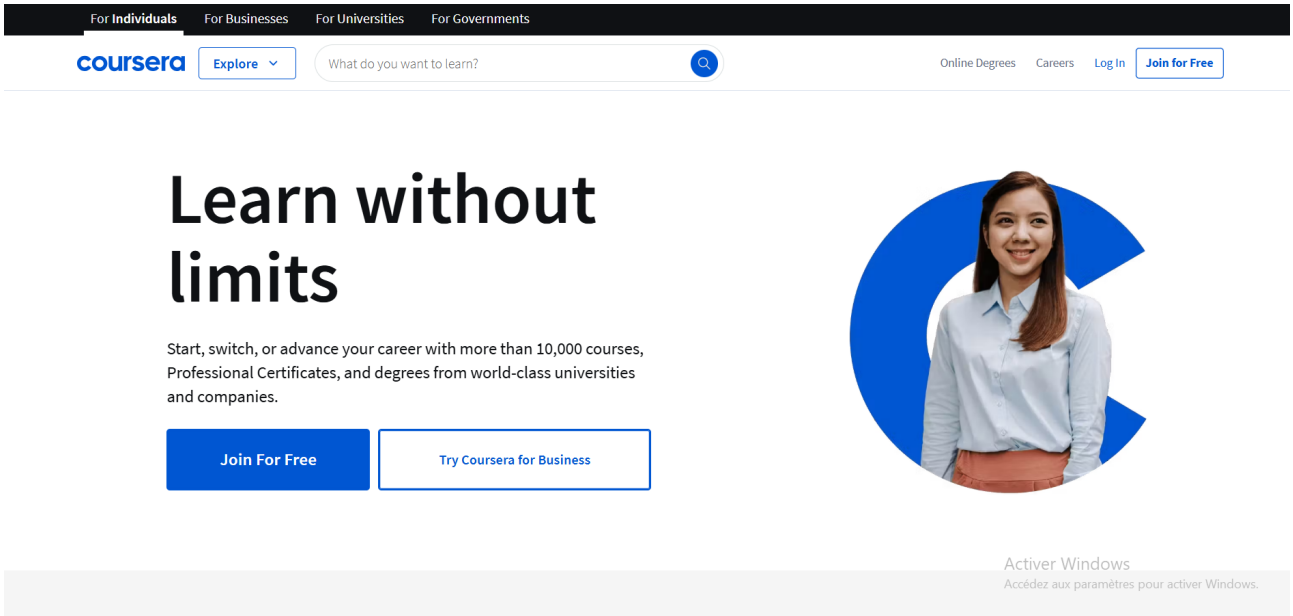


Figure 1.2: interface of the platform "Coursera" for individuals

The figure 1.3 represents the interface of Coursera web application for universities :

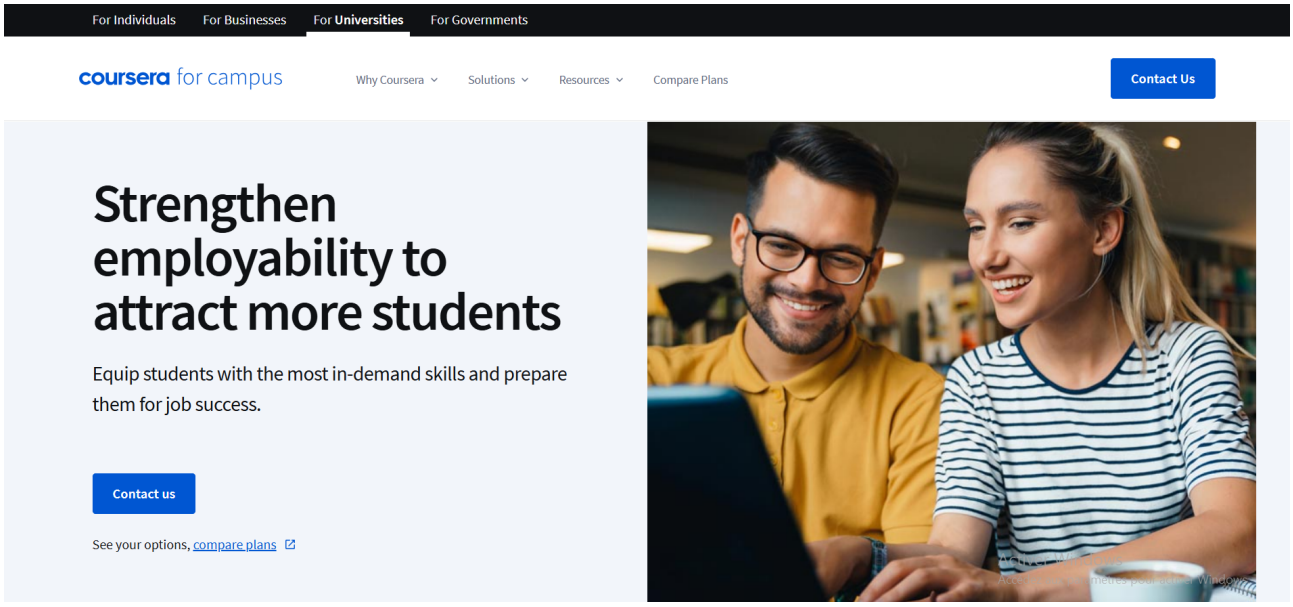


Figure 1.3: interface of the platform "Coursera" for universities

OpenClassrooms

OpenClassrooms is a French-based online learning platform that offers professional training and career support services.

- Website : `https://www.openclassrooms.com/`
- Technologies used : Ruby on Rails, JavaScript
- Year of Creation: 2013
- Number of Courses: Hundreds of courses and full career paths
- Number of Users: Over 3 million monthly users
- Mobile Application: Available
- Features for Users:
  - Mentorship-based learning.
  - Job guarantee on certain programs.
  - Recognized diplomas and certificates.
  - Project-based pedagogy.

The figure 1.4 represent the interface of OpenClassrooms :

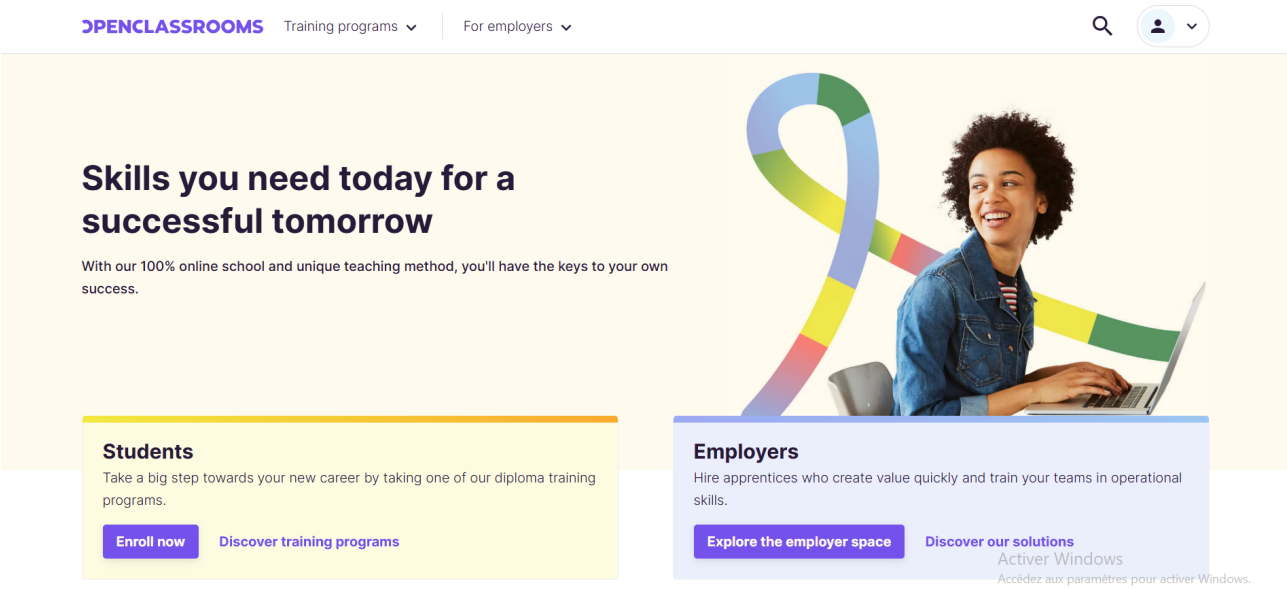


Figure 1.4: interface of the platform OpenClassrooms

1.6 Critique of Existing Educational Platforms

After analyzing the previously listed platforms and exploring their key features, we have identified several limitations that highlight the need for improvement in addressing the specific needs of learners, particularly in Algeria:

- Lack of Local and Cultural Contextualization
- Limited Accessibility
- Minimal Learner Support and Monitoring
- Insufficient Practical and Community Engagement

the table below : 1.1 represent a simple comparison between the three websites mentioned previously

Features	Udemy	Coursera	OpenClassrooms
Free Access	Partial	Partial	Partial
Certification	Paid	Paid/Free	Paid
Career-Oriented Programs	No	Yes	Yes
Practical Projects	Limited	Some	Yes
Career-Oriented Programs	no	Yes	Yes

Table 1.1: Comparative Table

1.7 Proposed Solutions

Based on the identified needs and shortcomings of existing platforms, we propose the development of SkillZone: a modern mobile application that combines structured learning paths, gamified progress tracking, and accessible content tailored to the local context. SkillZone focuses on accessibility, affordability, providing personalized dashboards, and a motivating environment tailored to young learners in Algeria.

1.8 Functional Requirement

Our platform is designed with two primary user types: the student and the course instructor.

1.8.1 For Students

- Create an account and log in securely
- Browse available courses
- Follow the course content
- Take quizzes at the end of the course
- Earn points by successfully completing quizzes
- Unlock premium courses after accumulating the required number of points
- Check the dashboard to track progress

1.8.2 For Teachers

- Create an account and log in securely
- Manage course : add, edit or delete courses
- Validate course content

1.9 Non-functional Requirements

Regarding non-functional requirements, they represent the objectives related to the aesthetics of system performance and its limitations in performance. They consist of specific goals that the system must have, so we decided that our mobile application should cover the following non-functional requirements:

- Our mobile application should have security where the user must be authenticated to access the site using a username and password
- The UX aspect should be organized so that our mobile application is understandable, easy to use, and enjoyable at the same time
- Ensuring technical support to solve technical problems or respond to user questions
- Ensuring the continuity of the mobile application and its potential for better future development.

1.10 Development Constraints

1.10.1 Technology to use

- **Design:** Figma , Canva
- **Front-end:** flutter Dart Framework, HTML, CSS, React JS
- **Back-end:** Django Rest Framework (DRF)
- **LaTex:** to write the report
- **Database:** PostgreSQL

1.10.2 The project duration

Globally we will not take more than 11 weeks starting from 6 February 2025.

1.11 Project Management and Task Planing (globaly)

For the implementation of our project and after analyzing client needs, we have defined the functionalities to develop and planned them into main tasks using the Gantt chart, here is this tasks:

- Research and Planning
- Market Research and Requirements Gathering
- Design and Prototyping
- Implementation
- testing and Quality Assurance
- Maintenance and Updates

This figure 1.5 represent our Gantt Chart :

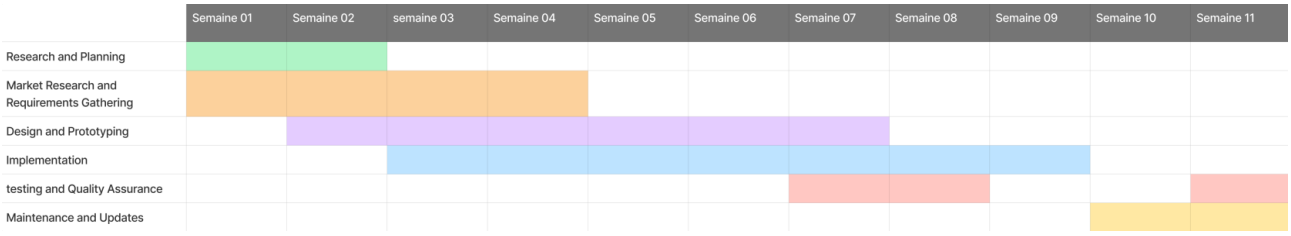


Figure 1.5: Gant Chart

1.12 Conclusion

This preliminary study has highlighted the growing demand for accessible, contextualized, and engaging online learning platforms. By analyzing existing platforms and identifying their limitations, we have confirmed the relevance of developing SkillZone as a solution tailored to the needs of learners. With a clear vision, functional and non-functional requirements, and a user-centered approach, SkillZone positions itself as an innovative project to empower individuals through skill development. The next step will involve the detailed design and modeling of the system's components.

Chapter 2

Design and Modeling System Components

2.1 Introduction

The development of **Skillzone**, a mobile platform designed to promote interactive learning through gamification, requires a solid and well-structured design phase. In this chapter, we present the technical foundation of the system by modeling its main components and outlining the logic behind their interaction. Building on the objectives and functional specifications defined earlier, we identify the different actors of the platform and analyze their roles. Through diagrams such as use case, sequence, class, and global architecture, we visualize the system's behavior and structure. Additionally, we define the data models and database structure to support both soft and hard skill learning features. This chapter serves to align the technical vision with the project's goals, ensuring that the transition from design to implementation is smooth, coherent, and scalable .

2.2 Identification of the actors

In the context of the Skillzone application, actors represent the different user types who interact directly with the system. Each actor has specific roles and responsibilities that contribute to the overall functionality of the platform. For this project, we have identified two primary actors: the **Student** and the **Teacher** .

Student

The student is the central user of the platform. Their role is to register, access courses, complete interactive quizzes, and accumulate points through learning activities. Students can unlock premium content using earned points or subscriptions, monitor their progress, and stay motivated through a reward-based system. They are the main beneficiaries of the platform's educational value.

Student Interactions

- **Account Management**
  - Registers and logs into the application.
  - Manages personal profile and preferences

- **Learning Content Access**
  - Views and browses available courses.
  - Accesses soft skills courses for free.
  - Unlocks hard skills courses using collected points or through a subscription.
- **Interactive Learning**
  - Participates in quizzes to validate knowledge.
  - Accumulates points based on quiz results and learning activities.
  - Tracks progress and achievements via a personal dashboard.
- **Motivation and Rewards**
  - Receives rewards and badges based on milestones.
  - Uses points as currency to access advanced content.

Teacher

The Teacher is responsible for managing the platform's backend content and user data. This includes uploading and organizing educational materials, monitoring user activities, managing course accessibility, and overseeing system performance. The Teacher ensures that the platform runs smoothly and that content is updated and aligned with learning objectives.

Teacher Interactions

- **Content Management**
  - Add, edit, and delete course content (videos, quizzes, descriptions).
  - Organizes courses into categories (soft skills / hard skills).
- **User Management**
  - Monitors user registrations and activities.
  - Manages user roles, bans users if necessary, or resets passwords.
- **System Monitoring**
  - Tracks platform performance and usage statistics.
  - Analyzes user engagement and feedback for continuous improvement.
- **Data Management**
  - Maintains and updates the database regularly.
  - Ensures data integrity and security compliance.

2.3 Use case diagram

The use case diagram for SkillZone provides a clear visualization of the system's functional requirements by illustrating the interactions between two primary actors—Student and Teacher—and the various actions they can perform within the platform.

Actors

Student: Engages in learning activities such as browsing available courses, following course content, taking quizzes, earning points, unlocking premium courses, and tracking progress through a personalized dashboard.

Teacher: Possesses all the capabilities of a student and additionally manages course content by adding, editing, or deleting courses.

Shared Use Case:

Authentication: Both students and teachers must sign up or log in to access the platform's features. This is a prerequisite for all other actions, ensuring secure access and personalized experiences.

Relationships:

include¿: Indicates mandatory functionality; for instance, all use cases include Authentication, emphasizing its necessity before any other interaction.

extend: Represents optional or conditional actions; for example, earning points extends to unlocking premium courses, highlighting that access to advanced content is contingent upon accumulated points or payment.

This diagram serves as a foundational tool in understanding the system's behavior from the perspective of end users and stakeholders, facilitating effective communication and guiding the development process.

And The figure 2.2 show us our global use case diagram of Skillzone app :

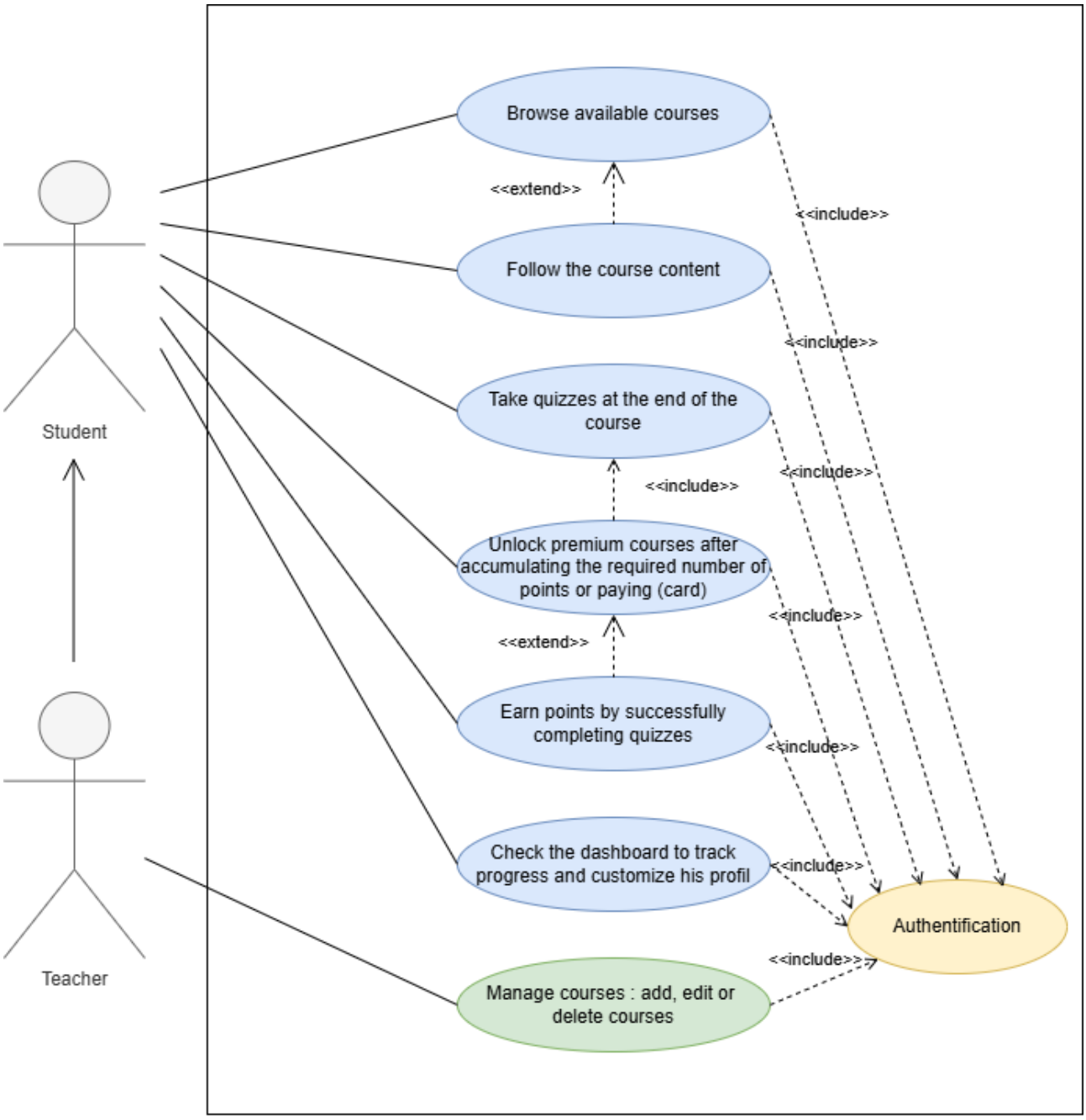


Figure 2.1: Our use case diagram

2.4 Global architecture diagram :

A Global Architecture Diagram provides a high-level visualization of a software system's structural components and their interactions. The depicted architecture represents a modern educational platform featuring seven distinct layers: a **Client Layer** with Flutter mobile and web apps communicating via HTTPS/JSON; an **API Gateway** using Django REST Framework for request routing and token validation; a dedicated **JWT Auth Service** for authentication; an **Application Layer** containing Quiz, User Management, Achievement, and Course modules with email verification and media upload capabilities; a **Data Access Layer** through Django ORM integrating PostgreSQL database with email, push notification, and file storage services; the core **PostgreSQL Database**; and various **External Services**. This decoupled, modular architecture ensures secure (JWT/HTTPS), scalable, and maintainable system design suitable for evolving educational applications.

This figure 2.2 represents the Global architecture diagram :

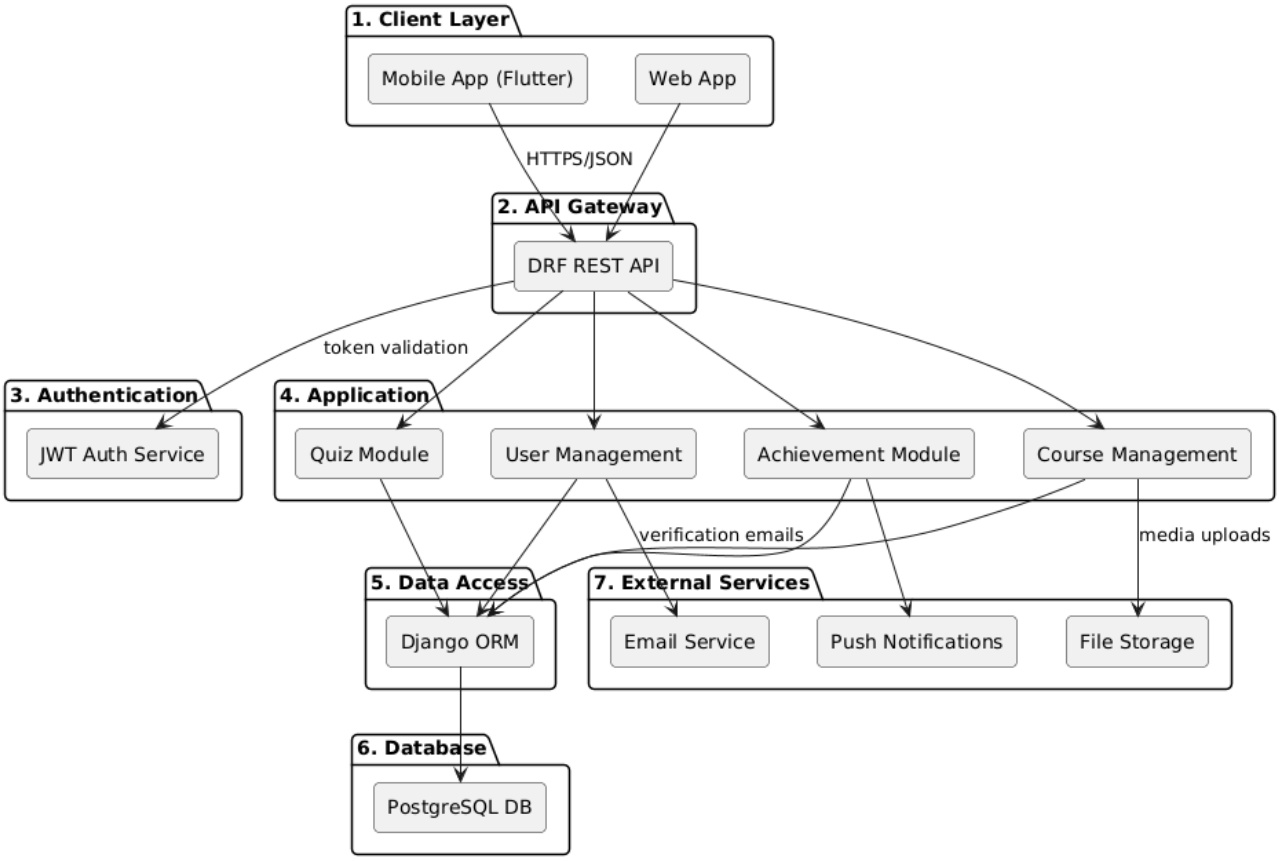


Figure 2.2: Global architecture Diagram

2.5 Sequence diagrams

A sequence diagram is a type of UML interaction diagram that shows how objects or components in a system interact through the exchange of messages over time. It emphasizes the chronological order of operations, making it useful for understanding the dynamic behavior of a system and verifying the flow of logic in different scenarios. Sequence diagrams are widely used to model system functionality, identify potential issues early in design, and ensure alignment with requirements.

In the case of the SKILLZONE application, the sequence diagram models key interactions such as user registration, email verification, login, course enrollment, and quiz completion. It clearly illustrates how the frontend communicates with various backend APIs like authentication, course, quiz, and achievement services. The diagram includes essential flows such as unlocking courses with points and awarding achievements. It highlights the role of JWT tokens in secure API requests. Email service integration is shown for verification purposes. The diagram also covers how user progress and scores are managed. Overall, it ensures that the system components work together seamlessly to deliver a smooth learning experience.

2.5.1 Sequence diagram "User registration"

This figure 2.3 represents the sequence diagram of the use case (User registration) :

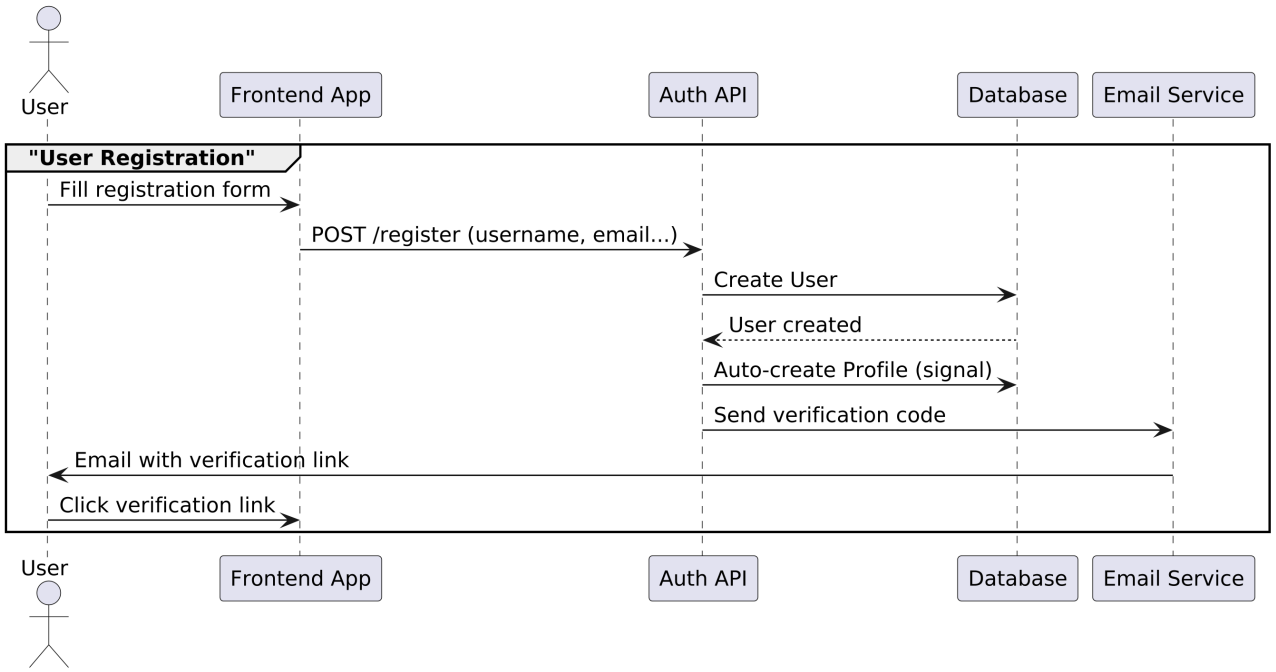


Figure 2.3: Sequence Diagram " User registration"

1. **The "User" actor initiates the process**
   - By filling out the registration form in the frontend application.
2. **The "Frontend App" processes the request**
   - Submits user data to the "Auth API" via `POST /register` (username, email, password).
3. **The "Auth API" handles account creation**
   - Creates a new user record in the database.
   - Receives confirmation ("User created") from the database.
   - Automatically generates a user profile (additional DB operation).
4. **Email verification is triggered**
   - The "Auth API" requests the "Email Service" to send a verification link.

- A time-sensitive verification link is emailed to the user.

5. **The "User" completes verification**
   - Clicks the verification link in the received email.
   - This action is captured by the frontend application.
6. **The system finalizes registration**
   - The "Frontend App" sends the verification token to "Auth API" (`POST /verify-email`).
   - The "Auth API" marks the email as verified in the database.
   - Returns a success response to the frontend.

2.5.2 Sequence Diagram "Autentication"

This figure 2.4 represents the sequence diagram of the use case (Autentication) :

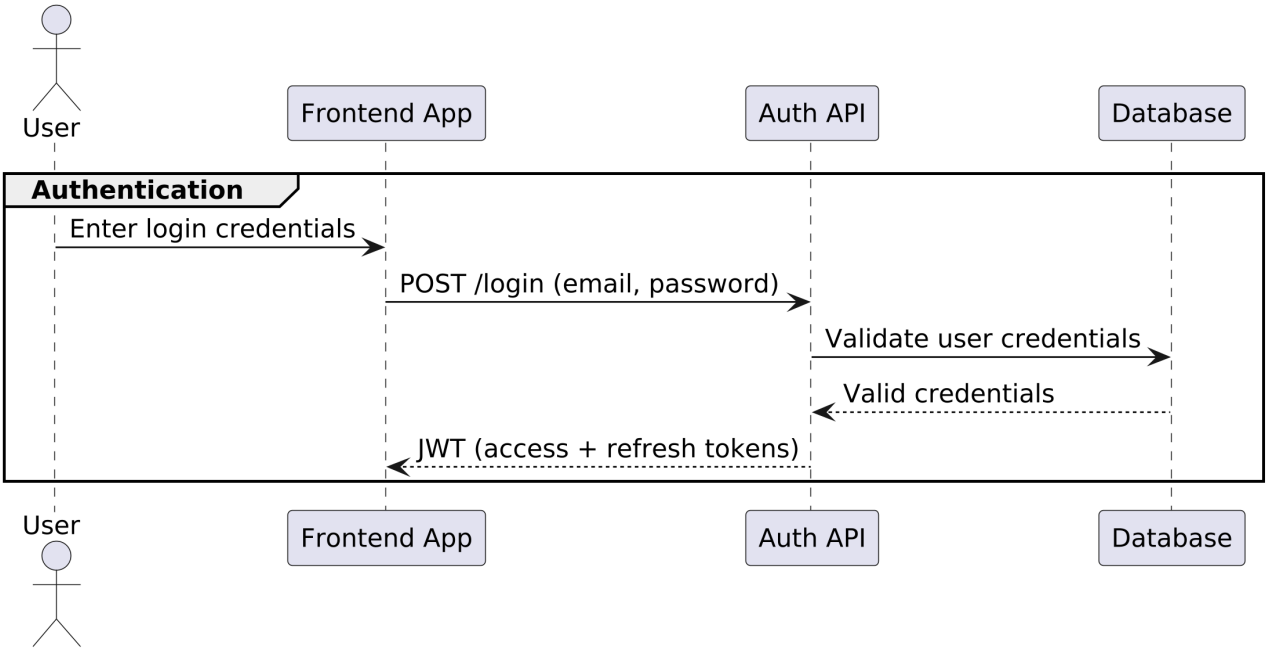


Figure 2.4: Sequence Diagram " Autentication"

1. **The "User" actor initiates the process**
   - By filling out the registration form in the frontend application.
2. **The "Frontend App" processes the request**
   - Submits user data to the "Auth API" via `POST /register` (username, email, password).
3. **The "Auth API" handles account creation**
   - Creates a new user record in the database.
   - Receives confirmation ("User created") from the database.
   - Automatically generates a user profile (additional DB operation).

4. **Email verification is triggered**
    - The "Auth API" requests the "Email Service" to send a verification link.
    - A time-sensitive verification link is emailed to the user.
5. **The "User" completes verification**
    - Clicks the verification link in the received email.
    - This action is captured by the frontend application.
6. **The system finalizes registration**
    - The "Frontend App" sends the verification token to "Auth API" (`POST /verify-email`).
    - The "Auth API" marks the email as verified in the database.
    - Returns a success response to the frontend.

2.5.3 Sequence diagram "Course Enrollment"

And This figure 2.5 represents this sequence diagram of the use case (Course Enrollment) :

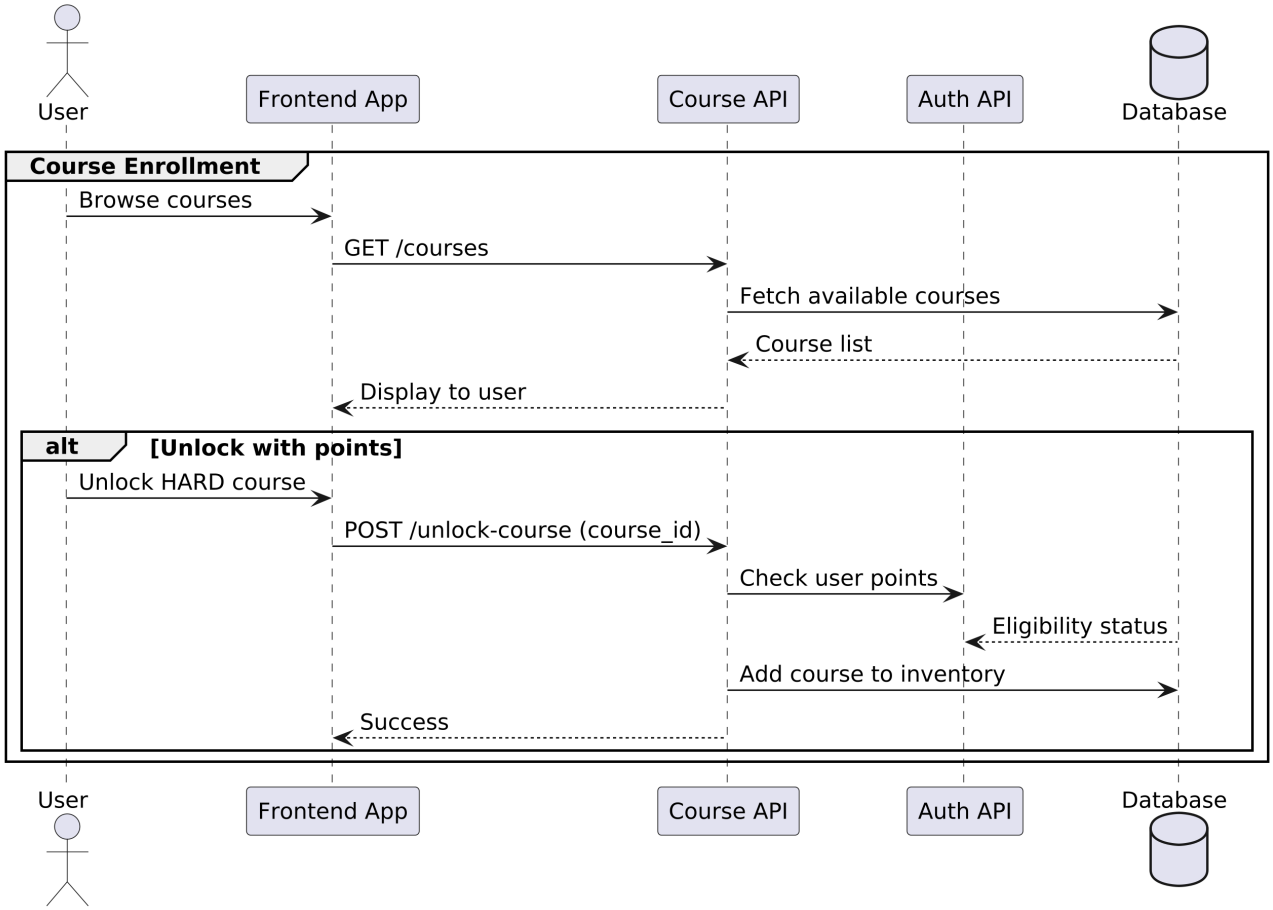


Figure 2.5: Sequence Diagram "Course Enrollment"

1. **The "User" browses courses**
   - Views available courses through the frontend interface.
2. **The "Frontend App" fetches courses**
   - Requests course list via `GET /courses` from "Course API".
   - Displays available courses to user.
3. **The "User" unlocks a premium course**
   - Selects a "HARD" course requiring points.
   - Frontend sends `POST /unlock-course` with course_id.
4. **The system verifies eligibility**
   - "Course API" checks user's points balance with "Auth API".
   - "Auth API" validates eligibility status with database.
5. **Successful enrollment**
   - If eligible, course is added to user's inventory.
   - Success confirmation returned to frontend.

2.5.4 Sequence diagram "API Request"

This figure 2.7 represents the sequence diagram of the use case (API Request) :

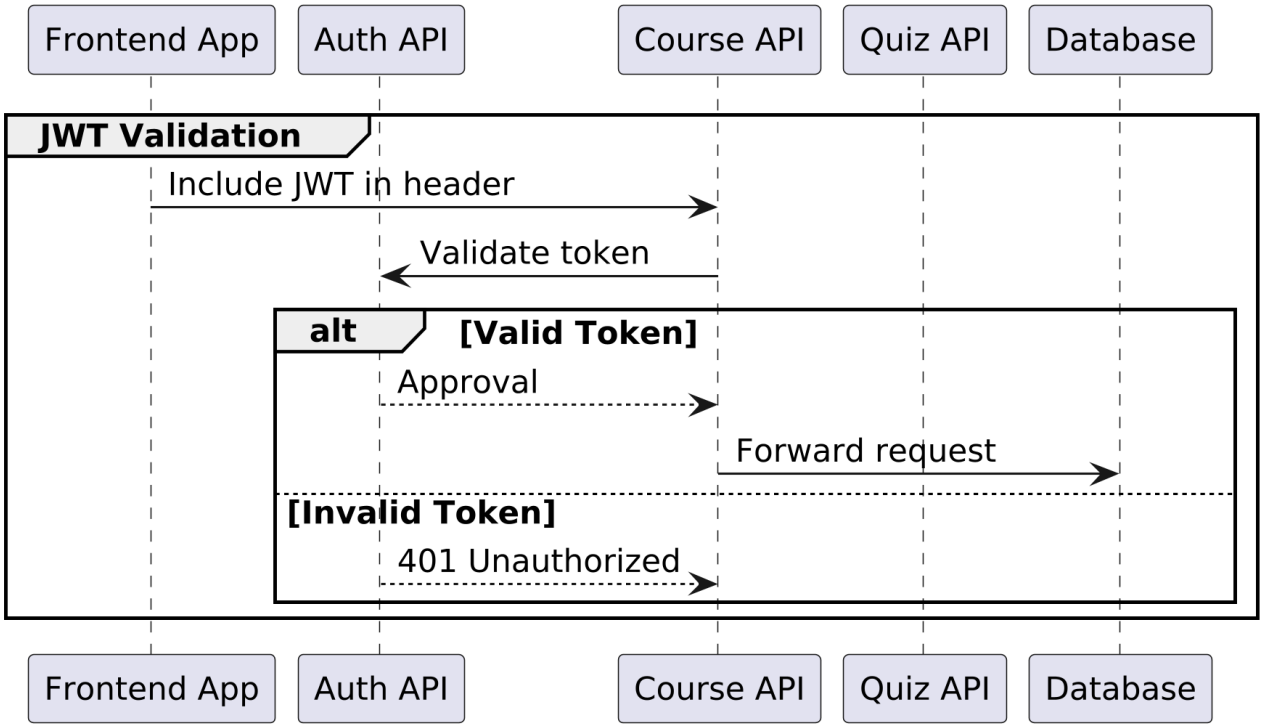


Figure 2.6: Sequence Diagram "API Request (JWT Validation)"

1. **The "Frontend App" initiates API request**
   - Includes JWT token in Authorization header.
   - Targets specific service (Course/Quiz API).
2. **Token validation process**
   - Receiving API forwards token to "Auth API" for validation.
   - Auth API verifies token signature and expiry.
3. **Validation outcome**
   - If valid: Auth API returns approval.
   - If invalid: Returns `401 Unauthorized`.
4. **Request processing**
   - For valid tokens: Original API processes request.
   - For invalid tokens: Error returned to frontend.

2.5.5 Sequence diagram "Quiz Completion"

This figure 2.7 represents the sequence diagram of the use case (Quiz Completion) :

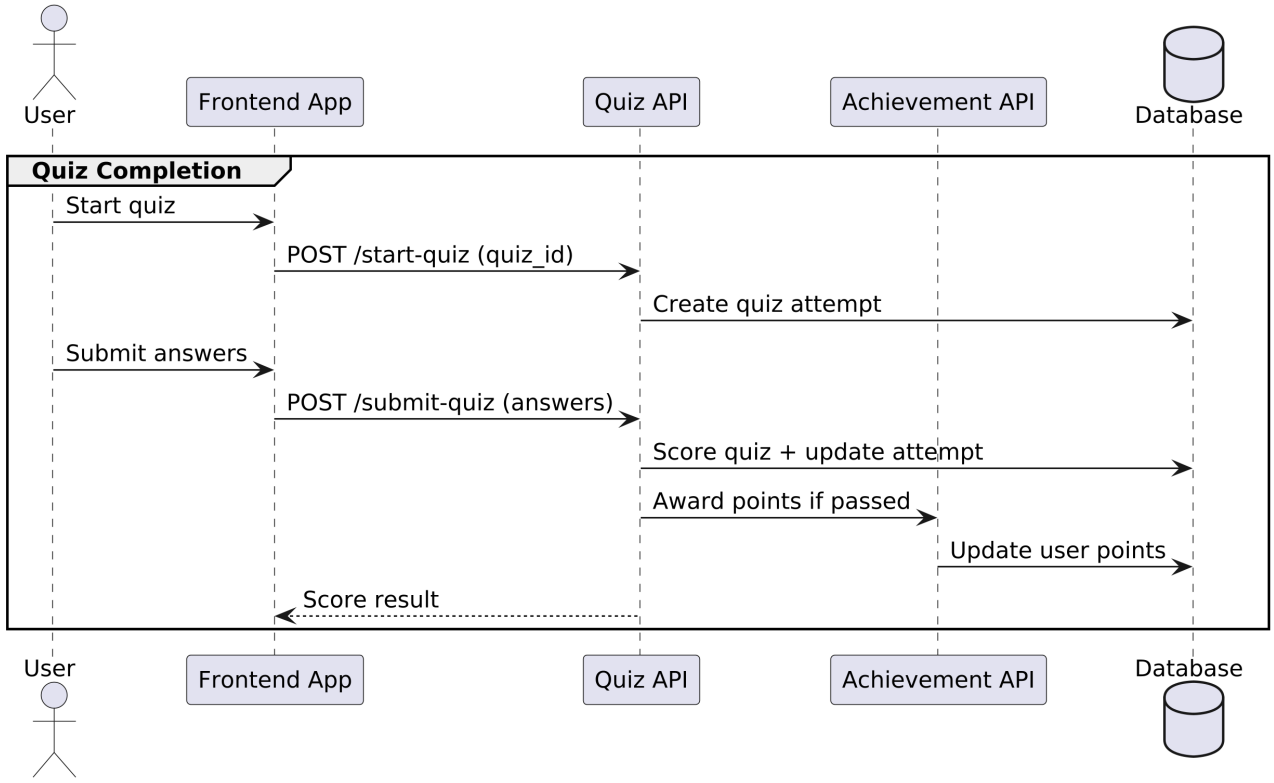


Figure 2.7: Sequence Diagram "Quiz Completion"

1. **Quiz Initiation**
   - User selects quiz in frontend interface
   - Frontend sends `POST /start-quiz` with quiz_id to Quiz API
   - Quiz API creates attempt record with:
     - Timestamp.
     - Initial status ("in-progress").
     - Time limit (if applicable).
2. **Answer Submission**
   - User submits answers through frontend.
   - Frontend sends `POST /submit-quiz` containing:
     - attempt_id.
     - Question responses (JSON format).
3. **Scoring & Evaluation**
   - Quiz API:
     - Validates attempt_id.
     - Scores answers against solution key.
     - Calculates percentage score.

- Updates attempt status ("completed").

4. **Reward Processing**

- For passing scores ($\geq$ passing threshold):
  - Quiz API notifies Achievement API.
  - Achievement API:
    * Awards points to user profile.
    * Checks achievement unlock conditions.
    * Updates database with new rewards.

5. **Result Delivery**

- Quiz API returns detailed results:
  - Score breakdown per question.
  - Time taken.
  - Points earned.
  - Unlocked achievements (if any).

2.5.6 Sequence diagram "Points and Achievement"

This figure 2.8 represents the sequence diagram of the use case (Points and Achievement) :

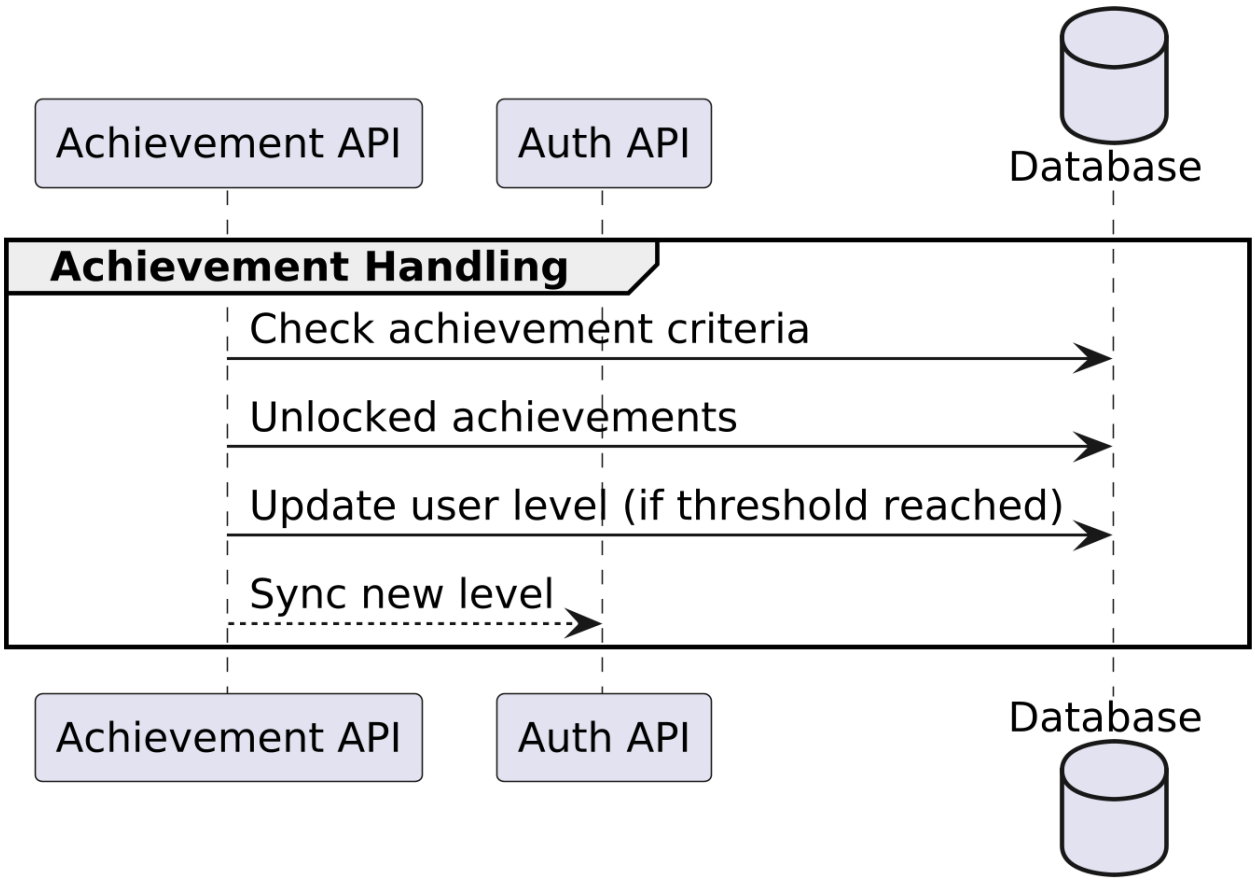


Figure 2.8: Sequence Diagram "Points and Achievement"

1. **Achievement Evaluation Trigger**
    - Initiated by system events:
        - Quiz completion.
        - Course milestones.
        - Login streaks.
2. **Criteria Verification**
    - Achievement API queries database for:
        - Current user points.
        - Existing achievements.
        - Threshold requirements.
3. **Achievement Unlocking**
    - For satisfied criteria:
        - New achievement record created.
        - Badge/image assigned to user profile.
        - Notification generated.
4. **Level Progression**
    - Achievement API checks total points against level thresholds
    - If level-up occurs:
        - User level incremented.
        - New privileges unlocked.
        - Auth API updated for permission changes.
5. **System Synchronization**
    - Achievement API syncs with Auth API to update user level cache.

2.6 Class diagram

This class diagram represents the static structure of the SkillZone educational platform. It illustrates the key classes in the system, such as User, Course, Quiz, and Achievement, along with their attributes and the relationships between them. This visual model captures how users interact with courses and quizzes, track their progress, and unlock achievements based on certain conditions. Each relationship is defined with cardinalities, highlighting how many instances of one class can be associated with another. Overall, this diagram provides a clear and organized overview of the system's data model and interactions .

And here 2.9 is our class diagram :

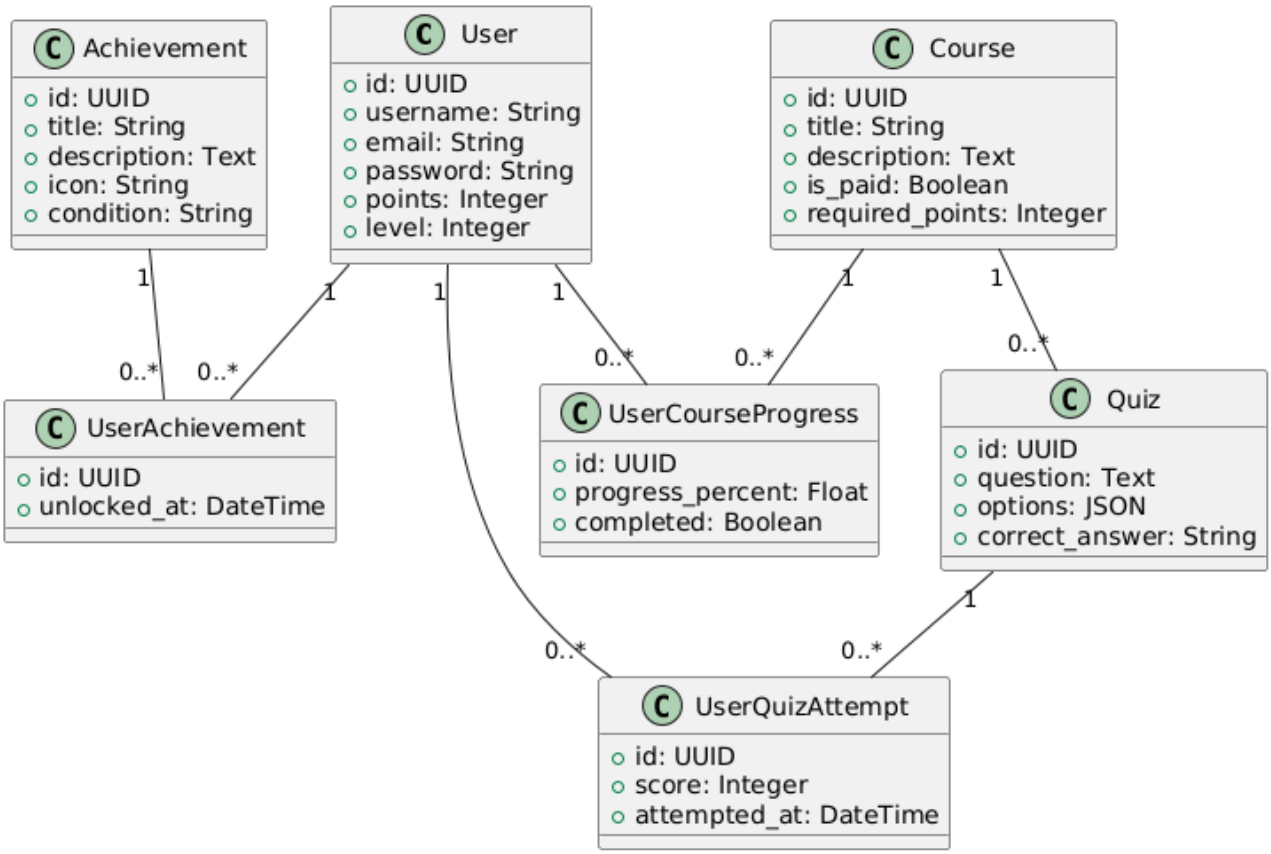


Figure 2.9: class diagram

2.6.1 Relational model

It is a conceptual framework used to organize and structure data in a database. This one allows for efficient storage, retrieval, and manipulation of data, making it a widely used model in database management systems.

And here is our relational model :

- **User** (user_id, username, email, password, points, level).
- **Course** (course_id, title, description, is_paid, required_points).
- **Quiz**(quiz_id, question, options, correct_answer, course_id).

- **Achievement**(achievement_id, title, description, icon, condition).
- **UserAchievement**(id, unlocked_at, user_id, achievement_id).
- **UserCourseProgress**(id, progress_percent, completed, user_id, course_id).
- **UserQuizAttempt**(id, score, attempted_at, user_id, quiz_id).

2.6.2 Data dictionary

This table 2.1 represents Data dictionary associated with our class diagram :

Class	Attribute	Type	Description
User	user_id	UUID	Primary Key, unique ID for each use
	username	String	Username of the user
	email	String	User's email address
	password	String	Encrypted password
	points	Integer	Points accumulated by the user
	level	Integer	User's current level
Course	course_id	UUID	Primary Key, unique ID for each course
	title	String	Title of the course
	description	Text	Course description
	is_paid	Boolean	Whether the course is paid or free
	required_points	Integer	Points needed to access the course
Quiz	quiz_id	UUID	Primary Key, unique ID for each quiz
	question	Text	The quiz question
	options	JSON	List of possible answers (e.g., A, B, C, D)
	correct_answer	String	The correct answer value
	course_id	UUID	Foreign Key referencing Course(course_id)
Achievement	achievement_id	UUID	Primary Key, unique ID for each achievement
	title	String	Title of the achievement
	description	Text	Achievement details
	icon	String	Icon representing the achievement
	condition	String	Condition to unlock the achievement
UserAchievement	id	UUID	Primary Key
	unlocked_at	DateTime	Date and time the achievement was unlocked
	user_id	UUID	Foreign Key referencing User(user_id)
	achievement_id	UUID	Foreign Key referencing Achievement(achievement_id)
UserCourseProgress	id	UUID	Primary Key
	$\text{progress}_p ercent$	Float	User's progress in the course (0–100%)
	completed	Boolean	Whether the course is completed
	user_id	UUID	Foreign Key referencing User(user_id)
	course_id	UUID	Foreign Key referencing Course(course_id)
UserQuizAttempt	id	UUID	Primary Key
	score	Integer	Score obtained in the quiz
	attempted_at	DateTime	Date and time of the quiz attempt
	user_id	UUID	Foreign Key referencing User(user_id)
	quiz_id	UUID	Foreign Key referencing Quiz(quiz_id)

Table 2.1: Data dictionary

2.7 Queries PostgreSQL with Examples :

PostgreSQL is a powerful, open-source relational database management system (RDBMS) known for its reliability, extensibility, and SQL compliance. It supports complex queries, indexing, transactions, and advanced data types. Below are some essential PostgreSQL queries with practical examples to help you interact with databases efficiently.

2.7.1 Register

This function is called when a user submits a registration form. It processes the registration data from the request payload (request.data), validates the input using UserRegistrationSerializer, and checks if the user accepted the terms of service. If validation passes, it securely creates a new user account in a database transaction, removing sensitive fields like passwords and terms acceptance before storage. The function then attempts to send a verification email to the user's provided address. If successful, it returns a JSON response confirming the registration was completed and email verification was initiated, including the user's email and account type (teacher/regular). If any step fails - whether due to invalid data, terms rejection, or email delivery issues - it returns appropriate error responses with status codes (400 for validation errors, 500 for server-side failures). The implementation uses Django's atomic transactions to ensure data consistency and follows security best practices by never exposing raw passwords in responses or logs.

This figure 2.10 represents The register function code :

```python
def register(request):
    serializer = UserRegistrationSerializer(data=request.data)
    if not serializer.is_valid():
        return Response({
            'status': False,
            'message': 'Validation error',
            'data': None,
            'errors': serializer.errors
        }, status=400)

    data = serializer.validated_data
    if not data.get('accept_terms'):
        return Response({
            'status': False,
            'message': 'Terms must be accepted',
            'errors': {'accept_terms': ['Terms must be accepted.']}
        }, status=400)

    with transaction.atomic():
        user_data = data.copy()
        password = user_data.pop('password')
        user_data.pop('password2', None)
        user_data.pop('accept_terms', None)
        is_teacher = user_data.pop('is_teacher', False)

        user = User.objects.create_user(password=password, **user_data)
        user.profile.is_teacher = is_teacher
        user.profile.save()

        try:
            send_verification_email(user)
        except Exception as e:
            return Response({
                "status": False,
                "message": "Registered but failed to send email",
                "errors": {"email": str(e)}
            }, status=500)

    return Response({
        'status': True,
        'message': 'Registration successful. Check email for verification.',
        'data': {
            'email': user.email,
            'requires_verification': True,
            'is_teacher': is_teacher
        }
    }, status=201)
```

Figure 2.10: Register Function Code

2.7.2 Start_quiz

This function manages the process of starting a quiz attempt for an authenticated user. When called, it first retrieves the requested quiz and checks whether the user has exceeded the maximum allowed attempts (if any restrictions are set). If an existing incomplete attempt is found,

the function calculates the remaining time and either resumes it if time remains or automatically marks it as failed if the time limit has expired. For new attempts, it creates a fresh QuizAttempt record with the full time allocation. The function then returns the attempt details including the attempt ID, remaining time (for both new and resumed attempts), and the quiz content itself - with questions randomized if the quiz is configured that way. Throughout this process, it handles various edge cases including attempt limits, time expiration, and concurrent attempts, while ensuring data integrity through proper model updates and providing clear response structures for the frontend to handle. The implementation uses Django's ORM for database operations and includes appropriate error handling through HTTP status codes.

This figure 2.11 The function start_quiz code :

```python
def start_quiz(request, quiz_id):
    """Start a new quiz attempt with randomization"""
    quiz = get_object_or_404(Quiz, id=quiz_id)


    if quiz.max_attempts > 0:
        attempt_count = QuizAttempt.objects.filter(
            quiz=quiz,
            user=request.user.profile
        ).count()
        if attempt_count >= quiz.max_attempts:
            return Response({
                "error": "Maximum attempts reached"
            }, status=status.HTTP_400_BAD_REQUEST)


    existing_attempt = QuizAttempt.objects.filter(
        quiz=quiz,
        user=request.user.profile,
        completed_at__isnull=True
    ).first()

    if existing_attempt:
        time_elapsed = timezone.now() - existing_attempt.started_at
        if time_elapsed.total_seconds() > quiz.time_limit:
            existing_attempt.is_passed = False
            existing_attempt.completed_at = timezone.now()
            existing_attempt.save()
        else:
            remaining_time = quiz.time_limit - int(time_elapsed.total_seconds())
            return Response({
                'attempt_id': existing_attempt.id,
                'remaining_time': remaining_time,
                'quiz': QuizDetailSerializer(quiz, context={'randomize': quiz.is_randomized}).data
            })


    attempt = QuizAttempt.objects.create(
        quiz=quiz,
        user=request.user.profile
    )

    return Response({
        'attempt_id': attempt.id,
        'remaining_time': quiz.time_limit,
        'quiz': QuizDetailSerializer(quiz, context={'randomize': quiz.is_randomized}).data
    })
```

Figure 2.11: Start_quiz Function Code

2.7.3 Statistics

This function retrieves and calculates achievement statistics for an authenticated user. It first fetches the user's earned achievements and compares them against the total available achievements in the system. The function then categorizes these achievements by type, calculating completion percentages for each category. Additionally, it identifies the user's five most recently earned achievements and computes the total points earned from all achievements. The response provides a comprehensive overview of the user's progress, including overall completion rates, breakdowns by achievement type, recent accomplishments, and cumulative points - all formatted as a structured JSON response. The implementation efficiently handles potential edge cases like division by zero and uses Django's ORM for optimized database queries.

This figure 2.12 The function Statistics code :

```python
def statistics(self, request):
    """Get achievement statistics for the user"""
    user_achievements = UserAchievement.objects.filter(user=request.user)
    total_achievements = Achievement.objects.count()
    earned_achievements = user_achievements.count()


    achievements_by_type = {}
    for achievement_type, _ in Achievement.TYPES:
        total = Achievement.objects.filter(type=achievement_type).count()
        earned = user_achievements.filter(achievement__type=achievement_type).count()
        achievements_by_type[achievement_type] = {
            'total': total,
            'earned': earned,
            'percentage': round((earned / total * 100) if total > 0 else 0, 1)
        }


    recent = UserAchievementSerializer(
        user_achievements.order_by('-earned_date')[:5],
        many=True
    ).data

    return Response({
        'total': total_achievements,
        'earned': earned_achievements,
        'percentage': round((earned_achievements / total_achievements * 100) if total_achievements > 0 else 0, 1),
        'by_type': achievements_by_type,
        'recent': recent,
        'points_earned': user_achievements.aggregate(
            total=Sum('achievement__points_reward')
        )['total'] or 0
    return Response(serializer.data)
```

Figure 2.12: Statistics Function Code

2.8 Conclusion

The design and modeling phase of SkillZone laid a robust foundation for the application's development by systematically addressing its structural and functional requirements. Through use case and sequence diagrams, we mapped user interactions, while the class diagram and relational model defined the system's data architecture. The global architecture ensured a scalable and secure framework, supported by PostgreSQL for efficient data management. This phase successfully bridged theoretical requirements with technical execution, setting a clear path for implementation. By clarifying roles, workflows, and system components, we established a cohesive blueprint to guide the next stages of development.

Chapter 3

Implementation

3.1 Introduction

In this chapter, we delve into the implementation of our platform, focusing on the development tools we've chosen and an outline of the application's functionalities. Our selection of development tools was driven by a commitment to modernity, cost-efficiency, and open-source accessibility.At the end we will introduce main and principal interfaces of Skillzone.

3.2 Development environment and tools

3.2.1 Work organization tools

3.2.1.1 Discord

Discord is a text, voice, and video-based online application that facilitates voice and video calls, media sharing, and text messaging. Initially developed and released for gamers, Discord has found use in many other scenarios such as friends chatting, online schooling, and more. It allows users to create groups with channels and rooms (salons) to stay organized and facilitate communication.

This 2.9 is where we organize our work in discord :

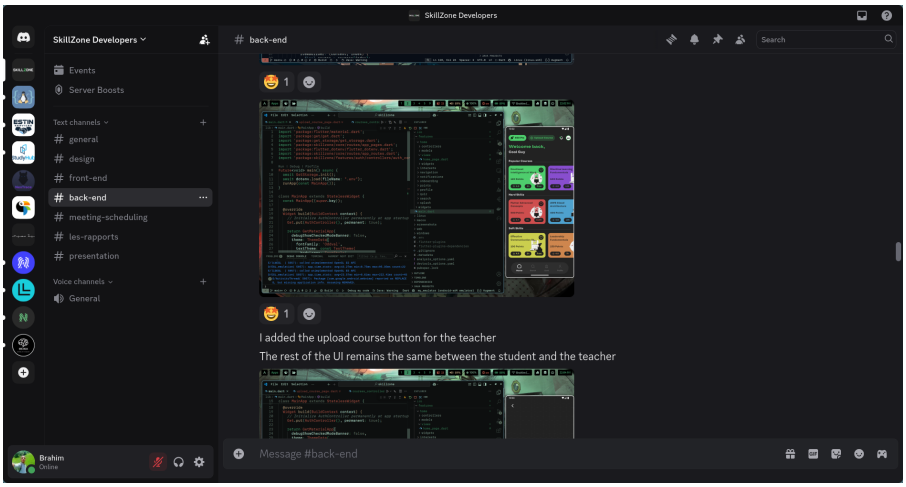


Figure 3.1: Discord

3.2.1.2 GitHub

GitHub is a for-profit company that offers a cloud-based Git repository hosting service. It makes it a lot easier for individuals and teams to use Git for version control and collaboration. GitHub's interface is user-friendly enough so even novice coders can take advantage of Git. It allows you to create, store, change, merge, and collaborate on files or code. Any member of a team can access and edit the GitHub repository and see the most recent version in real-time.

This figure 3.2 represent our work in GitHub :

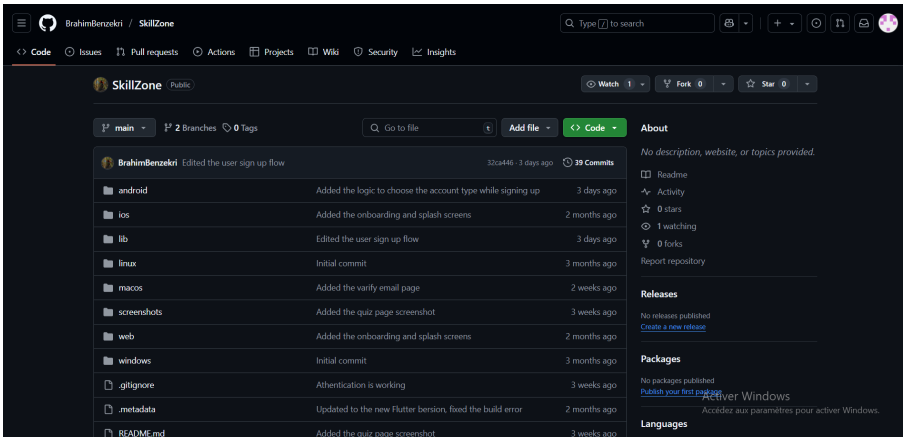


Figure 3.2: our GitHub workspace

3.2.2 Additional tools

3.2.2.1 VS (Visual Studio) Code

Visual Studio Code (VS Code) is a free and open-source code editor developed by Microsoft. It supports many programming languages and offers features like syntax highlighting, debugging, Git integration, and a built-in terminal. Originally designed for developers, it's now widely used in education, data science, and web development. Users can customize it with extensions and organize their work using folders and workspaces.

3.2.2.2 Draw.io

Draw.io (also known as diagrams.net) is a free, web-based diagramming tool used to create flowcharts, mind maps, UML diagrams, and more. It supports real-time collaboration and integrates with platforms like Google Drive, OneDrive, and GitHub. Originally built for visualizing technical and business processes, it's now used widely in education, project planning, and software design. Users can organize diagrams in layers and groups to keep their work clear and structured.

3.2.3 Design and Prototyping

3.2.3.1 Figma:

Figma is a cloud-based design and prototyping tool. It allows users to collaboratively create user interface (UI) and user experience (UX) designs for websites, mobile applications, and other digital products. Figma is widely used due to its ease of use, real-time collaboration features, and accessibility through web browsers.

3.2.4 Programming Languages

3.2.4.1 HTML (Hypertext Markup Language)

HTML is a markup language used to structure the content of web pages. It defines the layout of a document using tags that identify different types of content such as headings, paragraphs, lists, images, and links.

3.2.4.2 CSS (Cascading Style Sheets)

CSS is a style sheet language used to describe the presentation of an HTML document. It controls the appearance of elements on the page including font, color, size, and layout. CSS enables the separation of content from design, making web pages easier to maintain and update.

3.2.4.3 JavaScript

JavaScript is a programming language used to make web pages interactive and dynamic. It is commonly employed to add features like animations, interactive forms, and real-time content updates.

3.2.4.4 Flutter

Flutter is an open-source UI software development kit created by Google. It can be used to develop cross platform applications from a single codebase for the web,Fuchsia, Android, iOS, Linux, macOS, and Windows.Flutter is used internally by Google in apps such as Google Play and Google Earth as well as other software developers including ByteDance and Alibaba.

3.2.4.5 Django

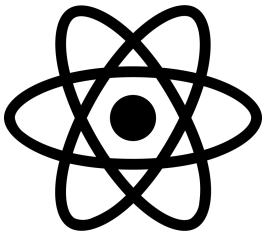
Django is a high-level Python web framework that enables rapid development of secure and scalable web applications using the model-template-view (MTV) pattern. It includes built-in tools for routing, database management, authentication, and an admin panel.

3.2.5 Front-end Technologies and frameworks

3.2.5.1 Dart

Dart is a programming language designed by Lars Bak and Kasper Lund and developed by Google. It can be used to develop web and mobile apps as well as server and desktop applications. Dart was mainly used for the development of the front-end of Skillzone.

3.2.5.2 React



React (also known as React.js) is a free and open-source front-end JavaScript library that aims to make building user interfaces based on components more "seamless". It is maintained by Meta (formerly Facebook) and a community of individual developers and companies. We used React for the front-end development of the website of Skillzone

3.2.5.3 Libraries Used in the SkillZone App Front-End (Flutter)

We have used the following additional packages and libraries:

Main Dependencies

- **auto_size_text** (^3.0.0): Automatically adjusts font size to fit within its bounds.
- **dio** (^5.8.0+1): Powerful HTTP client for making API requests.
- **flutter_dotenv** (^5.2.1): Loads environment variables from a `.env` file.

- **flutter_native_splash** (^2.4.4): Customizes native splash screens for Android and iOS.
- **flutter_svg** (^2.0.17): Renders SVG images.
- **get** (^4.7.2): Lightweight and powerful state management, routing, and dependency injection.
- **smooth_page_indicator** (^1.2.1): Adds customizable page indicators for page views.
- **percent_indicator** (^4.2.3): Displays linear and circular percent indicators.
- **video_player** (^2.9.3): Plays video files from the network, assets, or file system.
- **chewie** (^1.9.2): Provides a better and customizable video player UI (built on top of `video_player`).
- **video_player_android** (^2.8.2): Android-specific implementation of the video player.
- **get_storage** (^2.1.1): Local key-value storage solution compatible with the `get` package.
- **file_picker** (^10.1.2): Opens file explorer and lets users pick files from their device.
- **image_picker** (^1.1.2): Allows users to select images from their gallery or camera.

Development Dependencies

- **flutter_test**: Framework for writing Flutter unit tests.
- **flutter_lints** (^4.0.0): Recommended set of linter rules to maintain code quality.

Figure 3.3 illustrates the pubspec.yaml file located at the root of our project, where some frameworks, packages, and libraries are listed.

```yaml
name: skillzone
description: "A new Flutter project."
publish_to: 'none'
version: 0.1.0

environment:
  sdk: ^3.5.4

dependencies:
  flutter:
    sdk: flutter
  auto_size_text: ^3.0.0
  dio: ^5.8.0+1
  flutter_dotenv: ^5.2.1
  flutter_native_splash: ^2.4.4
  flutter_svg: ^2.0.17
  get: ^4.7.2
  smooth_page_indicator: ^1.2.1
  percent_indicator: ^4.2.3
  video_player: ^2.9.3
  chewie: ^1.9.2
  video_player_android: ^2.8.2
  get_storage: ^2.1.1
  file_picker: ^10.1.2
  image_picker: ^1.1.2

```

Figure 3.3: The pubspec.yaml file of our project

3.2.5.4 Libraries Used in the SkillZone Website Front-End (React)

We have used the following additional packages and libraries:

Main Dependencies

- **react** (^19.1.0) : Core React library for building user interfaces.
- **react-dom** (^19.1.0) : Enables React to interact with the browser DOM.
- **react-router-dom** (^7.6.0) : Handles client-side routing in React apps.
- **framer-motion** (^12.11.0) : Animations library for smooth UI transitions.

Development Dependencies

- **vite** (^6.3.5) : Fast build tool and development server for modern web apps.
- **@vitejs/plugin-react** (^4.4.1) : Vite plugin to support React with fast refresh.
- **eslint** (^9.25.0) : Linter for identifying and fixing code issues.
- **@eslint/js** (^9.25.0) : ESLint rules for JavaScript.
- **eslint-plugin-react-hooks** (^5.2.0) : Enforces best practices with React Hooks.
- **eslint-plugin-react-refresh** (^0.4.19) : ESLint support for React Fast Refresh.
- **@types/react** (^19.1.2) : TypeScript type definitions for React.
- **@types/react-dom** (^19.1.2) : TypeScript type definitions for React DOM.
- **globals** (^16.0.0) : Provides global variables for ESLint environments.

Figure 3.3 illustrates the package.json file located at the root of our project, where the frameworks, packages, and libraries are listed.

```json
{
  "name": "skillzone",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "framer-motion": "^12.11.0",
    "react": "^19.1.0",
    "react-dom": "^19.1.0",
    "react-router-dom": "^7.6.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.25.0",
    "@types/react": "^19.1.2",
    "@types/react-dom": "^19.1.2",
    "@vitejs/plugin-react": "^4.4.1",
    "eslint": "^9.25.0",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.19",
    "globals": "^16.0.0",
    "vite": "^6.3.5"
  }
}
```

Figure 3.4: The package.json file of our project

3.2.6 Back-end Technologies

3.2.6.1 Rest

Django REST Framework (DRF) is a powerful and flexible toolkit for building Web APIs in Django. It simplifies the process of serializing data, handling HTTP requests, managing authentication, and enforcing permissions. DRF supports both function-based and class-based views, and integrates seamlessly with Django's ORM and authentication system.

3.2.6.2 JWT (JSON Web Token) authentification

JWT (JSON Web Token) authentication is a stateless, secure way to handle user authentication in Django APIs. Used with Django REST Framework (DRF), it issues a token on login, which is sent with each request for verification. It works well with Django's user model and is easily implemented using libraries like simplejwt.

3.2.6.3 CORS (Cross-Origin Resource Sharing)

CORS (Cross-Origin Resource Sharing) in Django allows your API to be accessed from other domains. It's essential when your frontend and backend are on different origins. You can handle CORS in Django easily using the django-cors-headers package, which lets you control which domains are allowed to access your API.

3.2.7 SkillZone API Endpoints (v1)

3.2.7.1 Users App

- `POST /register/` — Create a new user account
- `POST /login/` — Authenticate and get JWT tokens
- `POST /logout/` — Blacklist refresh token
- `GET /profile/` — Get user profile details
- `PUT /profile/update/` — Update user profile information
- `POST /profile/change-password/` — Change user password
- `POST /update-points/` — Update user points
- `POST /update-device-token/` — Update notification device token
- `POST /verify-email/` — Verify user email with a code

3.2.7.2 Courses App

- `GET /inventory/` — Get user's unlocked courses
- `POST /upload-course/` — Upload a new course (admin/teacher only)
- `GET /<course_id>/lessons/` — Get lessons for a specific course
- `POST /<course_id>/unlock/` — Unlock a course for the user
- `GET /<course_id>/statistics/` — Get course completion stats

3.2.7.3 Quizzes App

- `GET /courses/<course_id>/quizzes/` — Get quizzes for a course
- `GET /quizzes/<id>/` — Get quiz details
- `POST /quizzes/<id>/start/` — Start a quiz attempt
- `POST /quizzes/<id>/submit/` — Submit quiz answers
- `GET /quizzes/<id>/statistics/` — Get quiz stats

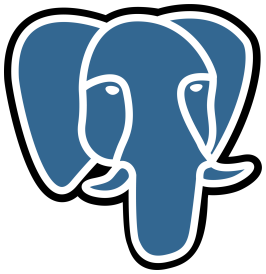
3.2.7.4 Achievements App

- `GET /` — List all achievements
- `GET /my_achievements/` — Get user's earned achievements
- `GET /available/` — Get achievements not yet earned
- `GET /<id>/progress/` — Get progress for a specific achievement
- `GET /leaderboard/` — Global achievement leaderboard
- `GET /statistics/` — Achievement stats for the user

3.2.7.5 Main App (skillzone)

- `GET /` — API root (endpoint info)
- `GET /api/v1/` — API v1 root
- `POST /token/refresh/` — Refresh JWT access token

3.2.8 POSTGRESQL Database Management



PostgreSQL is a powerful, open-source relational database management system (RDBMS) used to store and manage data. In Django, PostgreSQL can be easily integrated by configuring it in the settings.py file. It offers advanced features like ACID compliance, [**?**]. complex queries, and full-text search. Django uses the psycopg2 adapter to connect to PostgreSQL, enabling seamless integration with Django's ORM for data management.

3.2.9 PgAdmin4

PgAdmin4 is a popular, open-source graphical user interface (GUI) for managing PostgreSQL databases. It allows you to perform database tasks like creating tables, writing queries, managing users, and inspecting database objects visually. You can use pgAdmin 4 to connect to PostgreSQL databases locally or remotely and simplify database administration and management. [?].

3.3 Deployment Tools

3.3.1 Render

Render is a cloud platform that allows you to deploy and manage applications, including APIs, without worrying about infrastructure. It supports various languages and frameworks, including Django. For deploying Django APIs with Render, you can push your code to GitHub, link it to Render, and deploy with ease. Render handles scaling, security, and provides automatic HTTPS, making it an ideal platform for hosting Django-based APIs and web apps. [?]

3.3.2 Netlify

Netlify is a cloud platform designed primarily for deploying front-end applications and static sites,in our case, React websites. You can push your code to GitHub, link it to Netlify, and easily deploy. It handles builds, provides automatic HTTPS, continuous deployment, and serverless functions. [?]

3.3.3 Postman

Postman is a tool for testing and interacting with APIs. With Django APIs, you can use Postman to send requests (GET, POST, etc.), view responses, and debug your endpoints easily. [?].

For a full list of packages and dependencies used in the backend, refer to the **Skillzone Backend Repository**.

3.4 Principal Interfaces

3.4.1 Main pages

This figure 3.5 represents the interface of the home page:

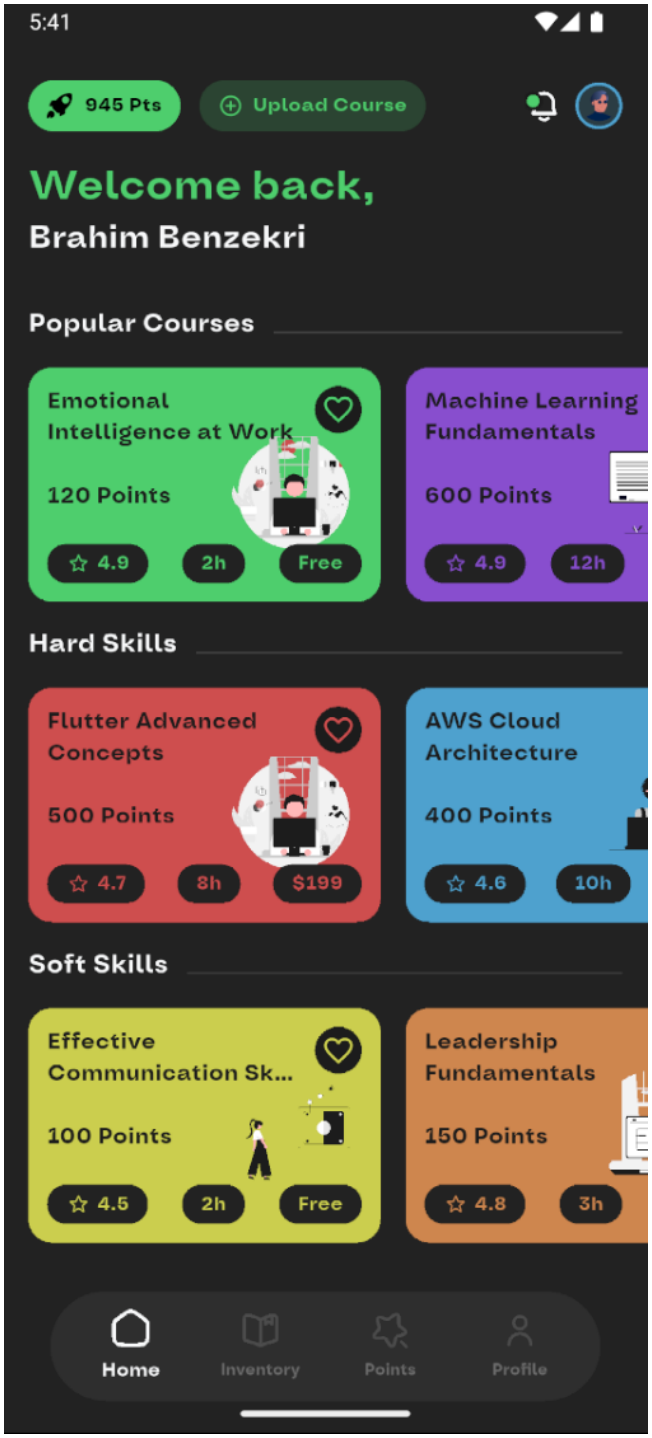


Figure 3.5: The home page

This figure 3.9 represents the interface of the inventory page where they find a list of their enrolled courses along with the liked ones:

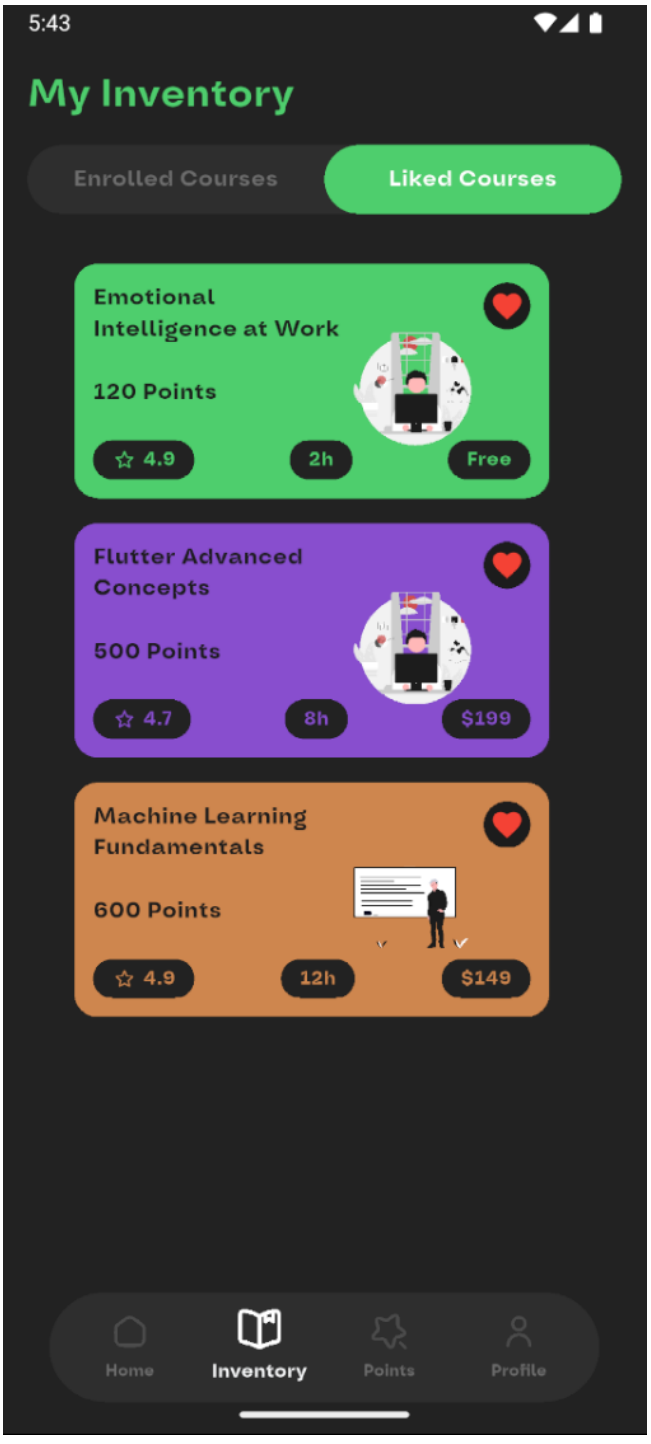


Figure 3.6: The inventory page.

This figure 3.7 represents the interface of the points gained by the student and his level :

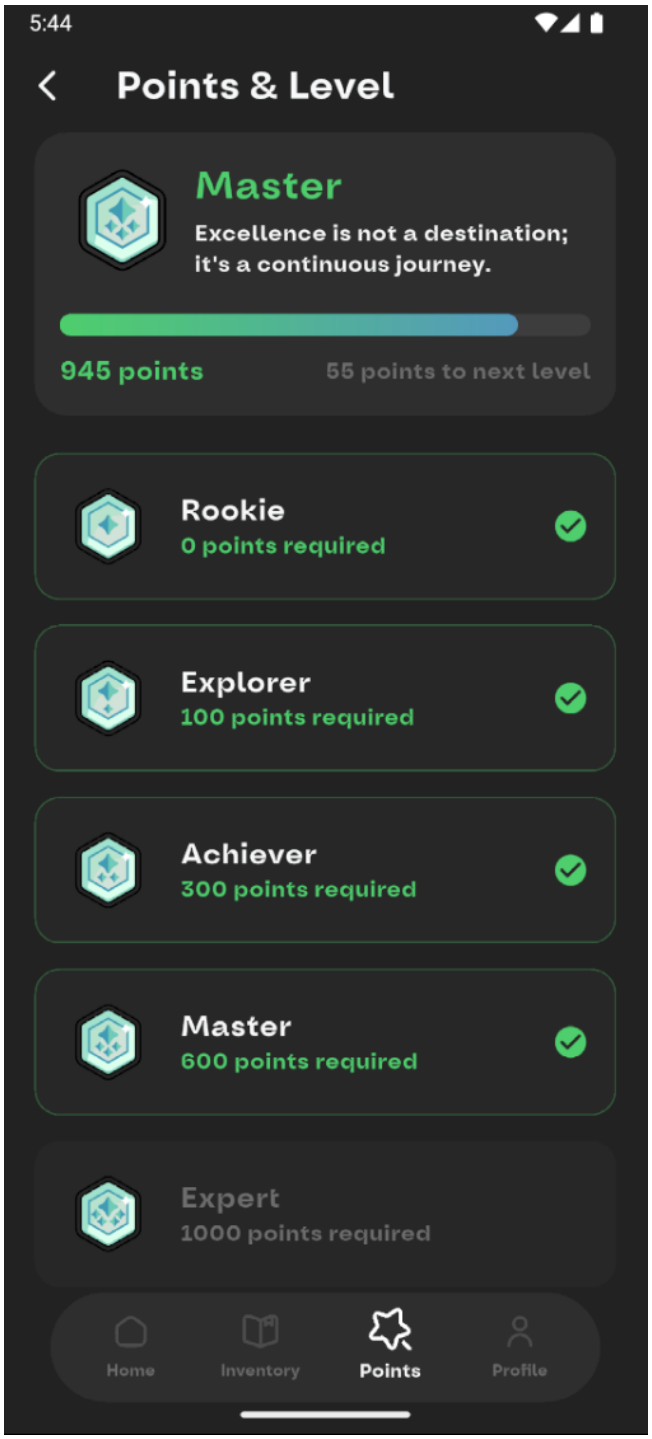


Figure 3.7: Points and level interface.

This figure 3.9 represents the interface of the student profile:

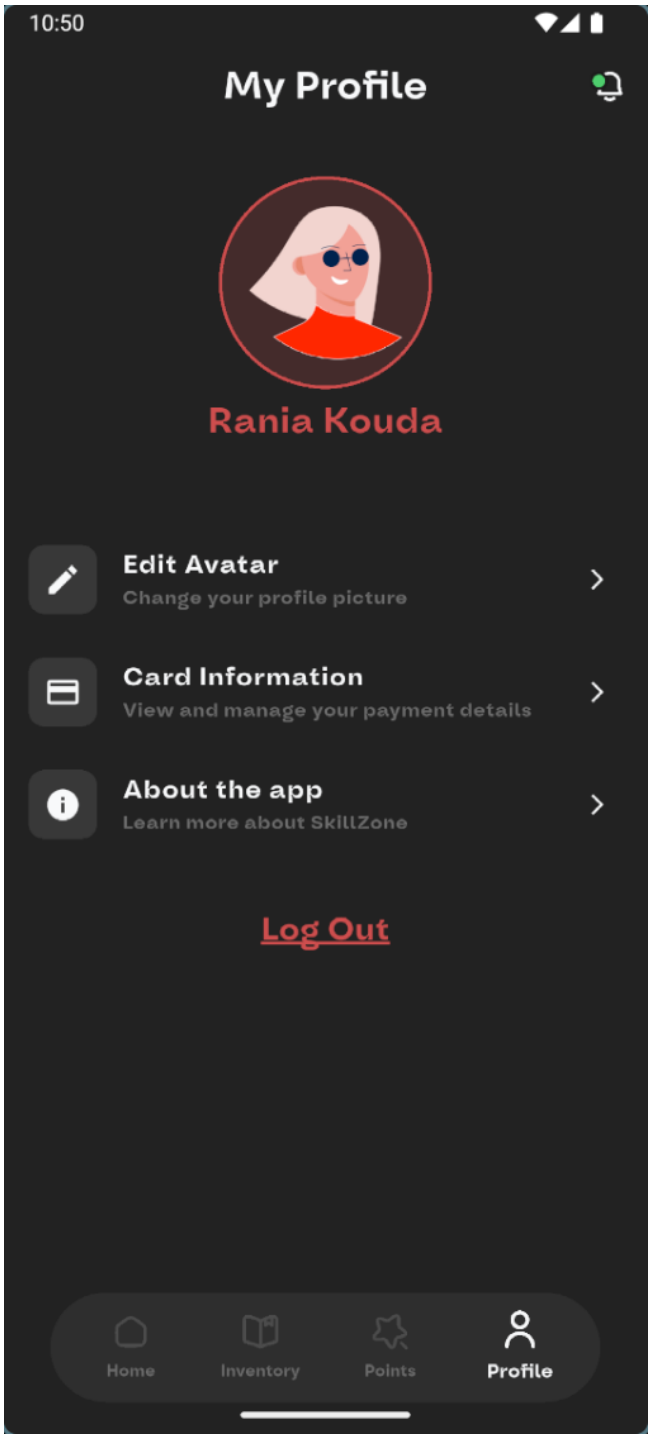


Figure 3.8: Interface of the student's profile.

This figure 3.9 represents the interface where the user can edit how his avatar looks in the app:

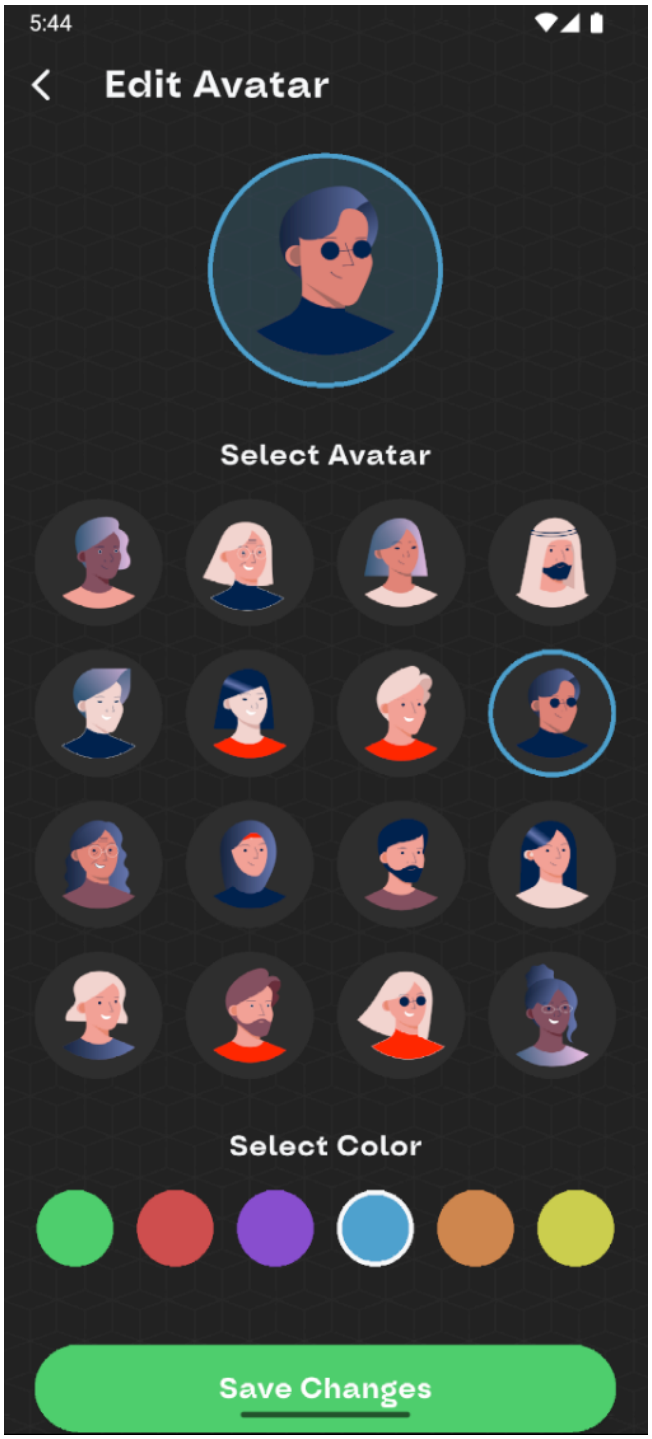


Figure 3.9: Edit avatar page.

3.4.2 For Teacher

This figure 3.10 represents the teacher profile interface:

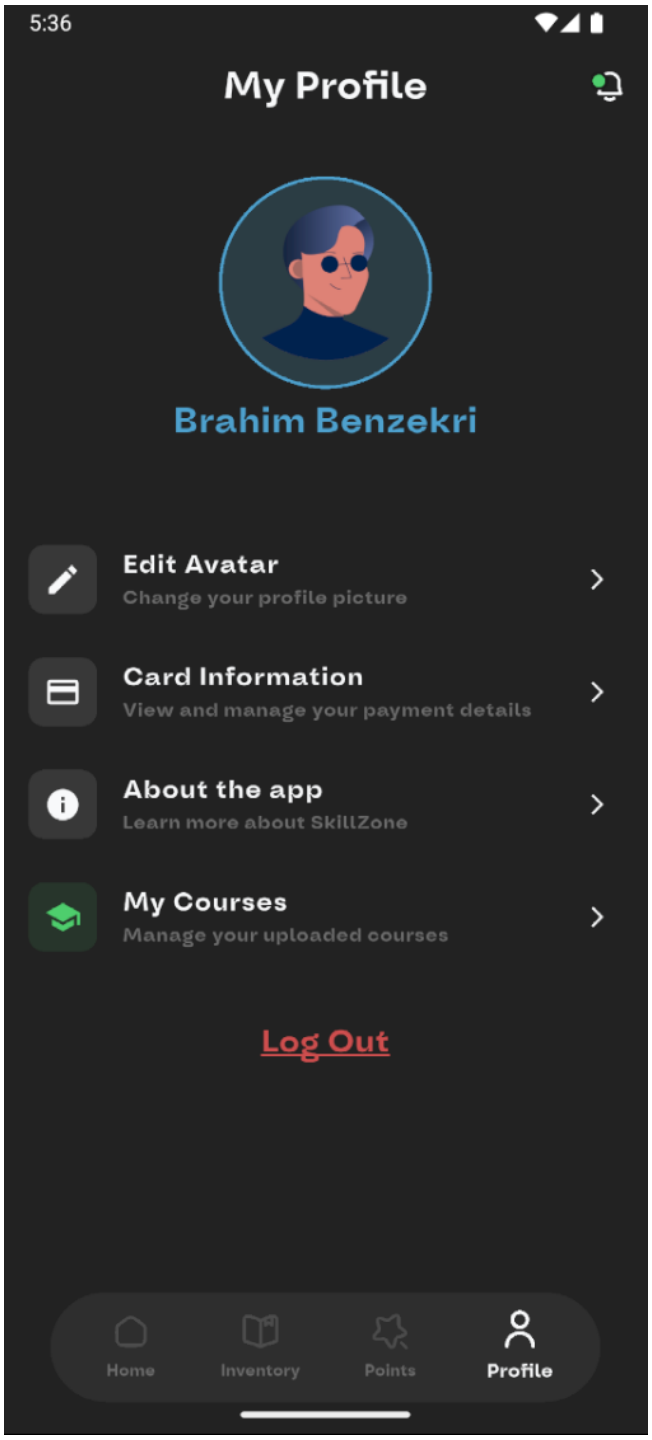


Figure 3.10: Interface of the teacher's profile.

This figure 3.11 represents The interface of uplaoding courses where the teacher can create, edit, and delete courses :

Figure 3.11: Teacher's interface for courses management.

This figure 3.12 Represents the interface for the courses the teacher already uploaded :

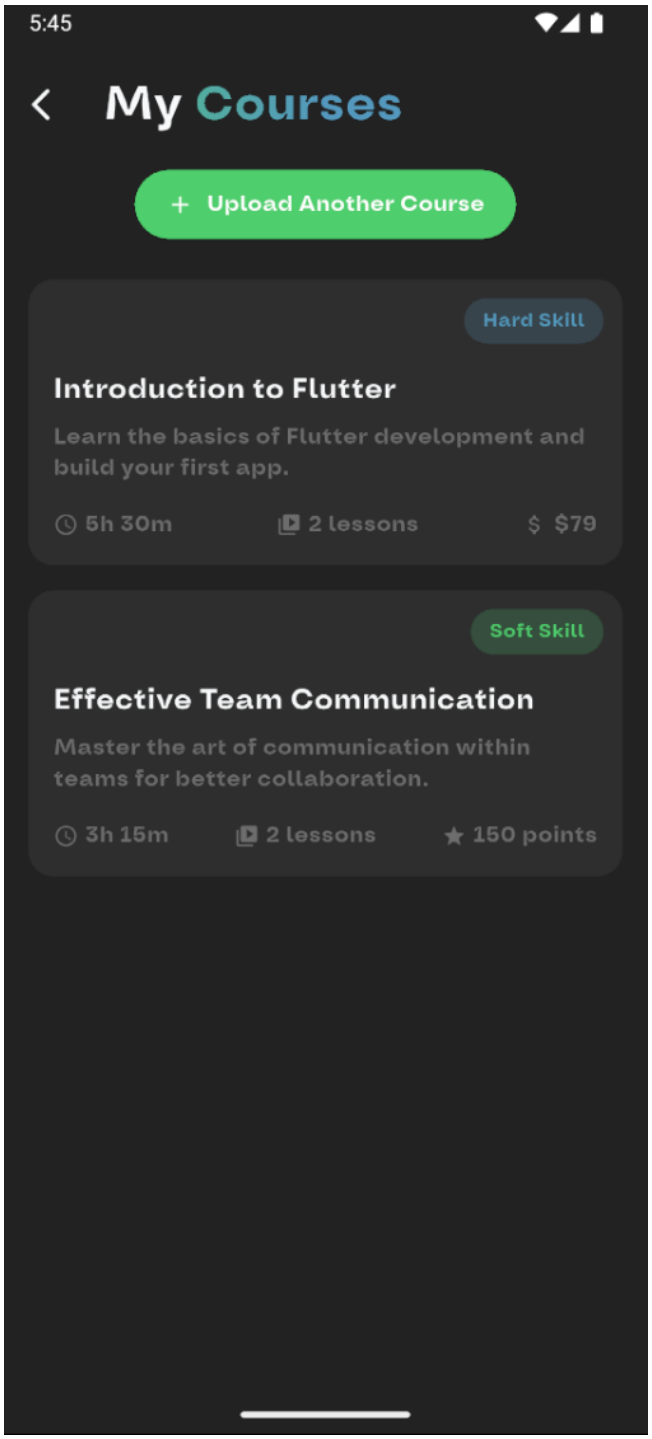


Figure 3.12: Teacher's interface for uploaded courses.

3.4.3 For Student

3.4.3.1 The available Courses

This figure 3.13 Represents the interface that shows the hard-skills courses :

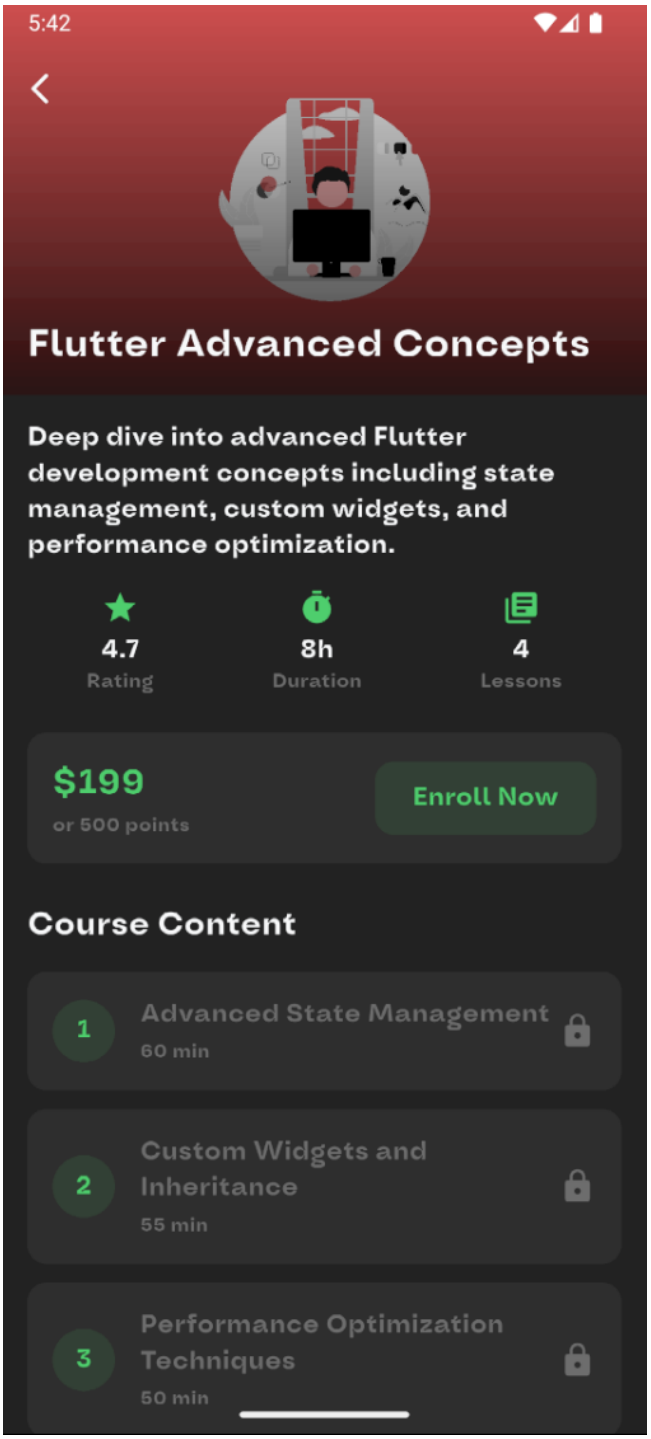


Figure 3.13: Hard-skills courses interface.

This figure 3.14 Represents the interface that shows the soft-skills courses :

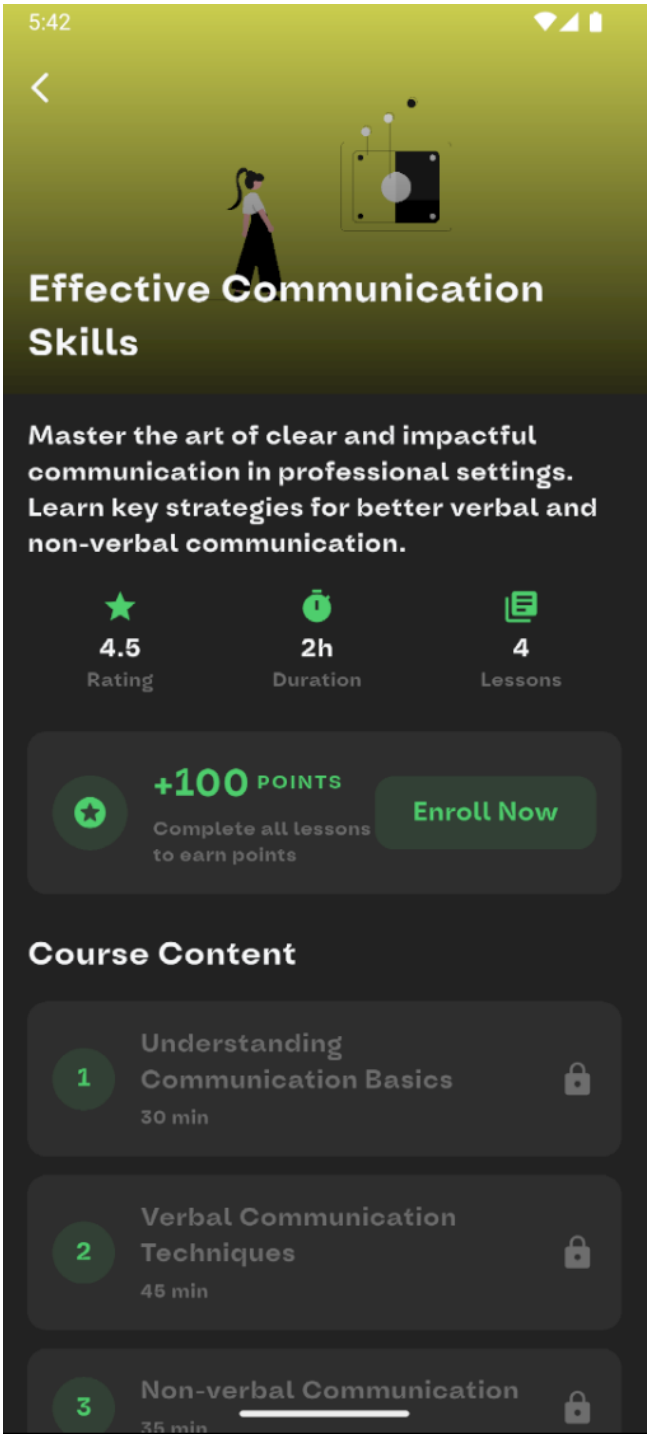


Figure 3.14: Soft-skills courses interface.

This figure 3.15 represents the interface that shows the space of the lesson's video :

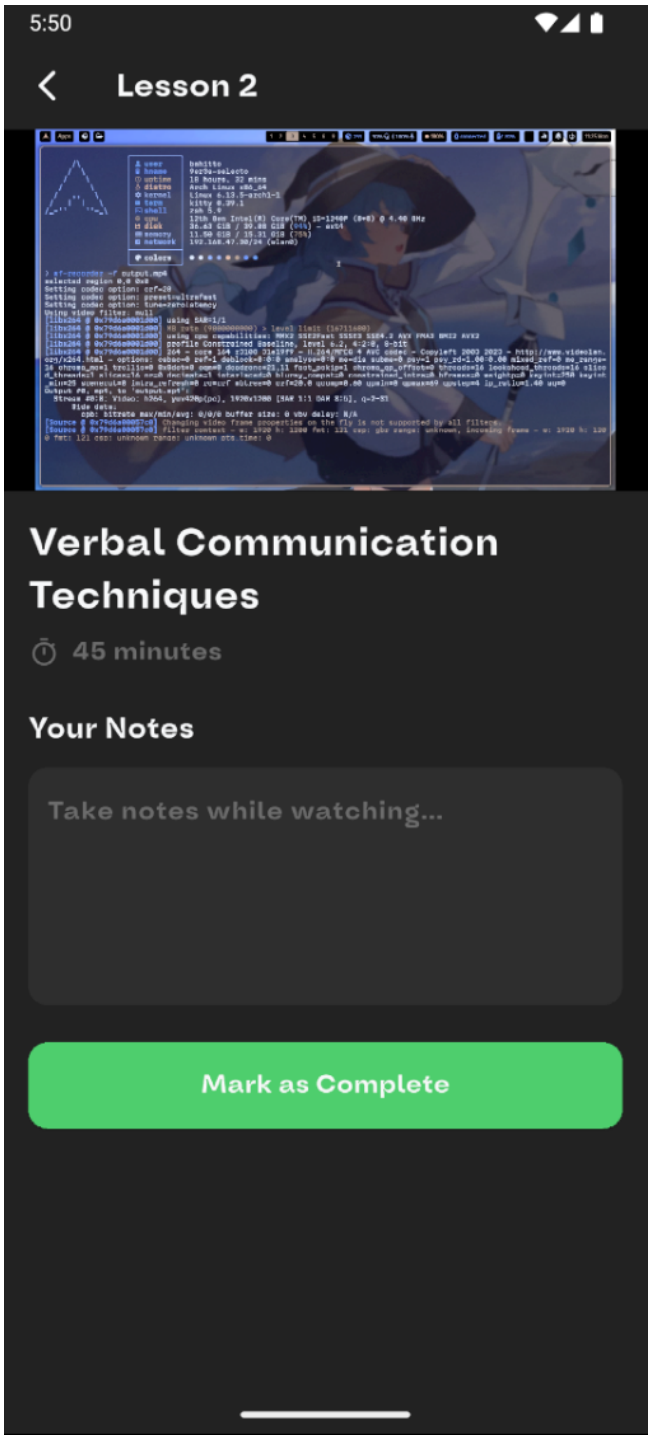


Figure 3.15: Lesson's video interface.

This figure 3.16 represents the interface that shows how the completed course looks like, where the user can take the quiz:

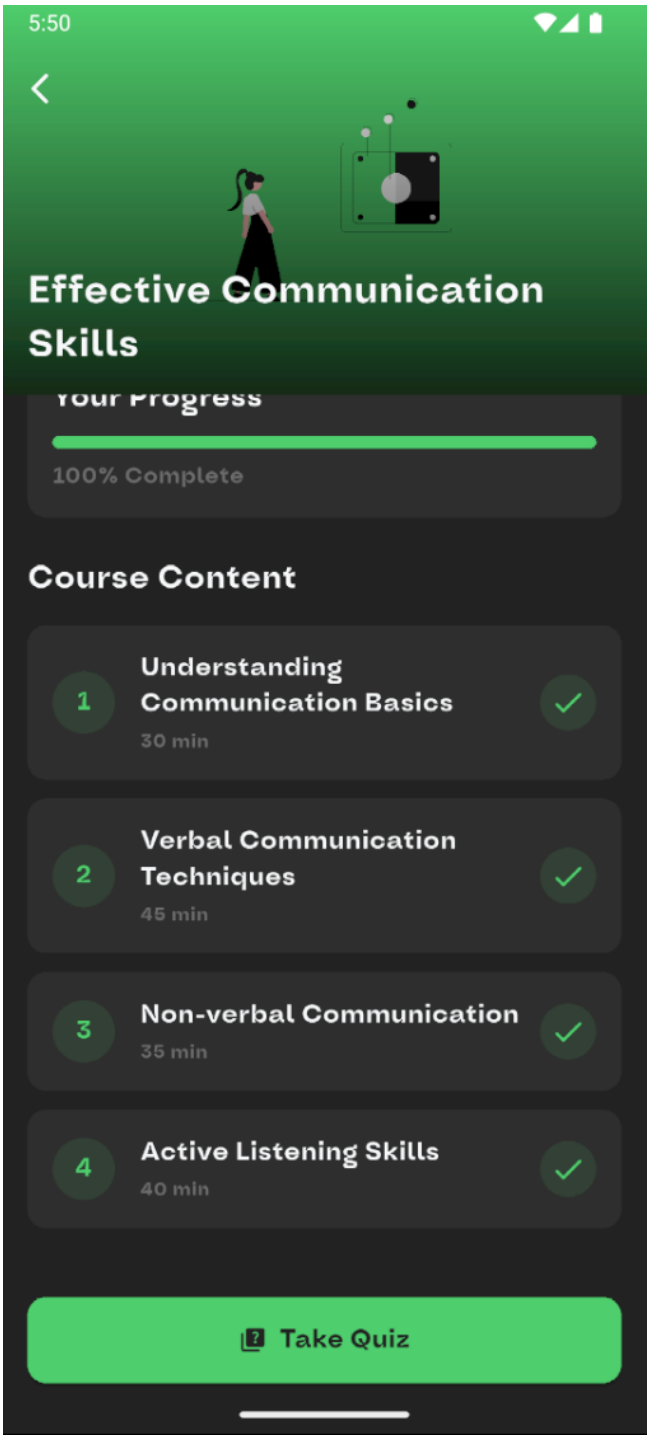


Figure 3.16: Completed course interface.

3.4.3.2 Quizzes

This figure 3.17 Represents the interface of Quizly :

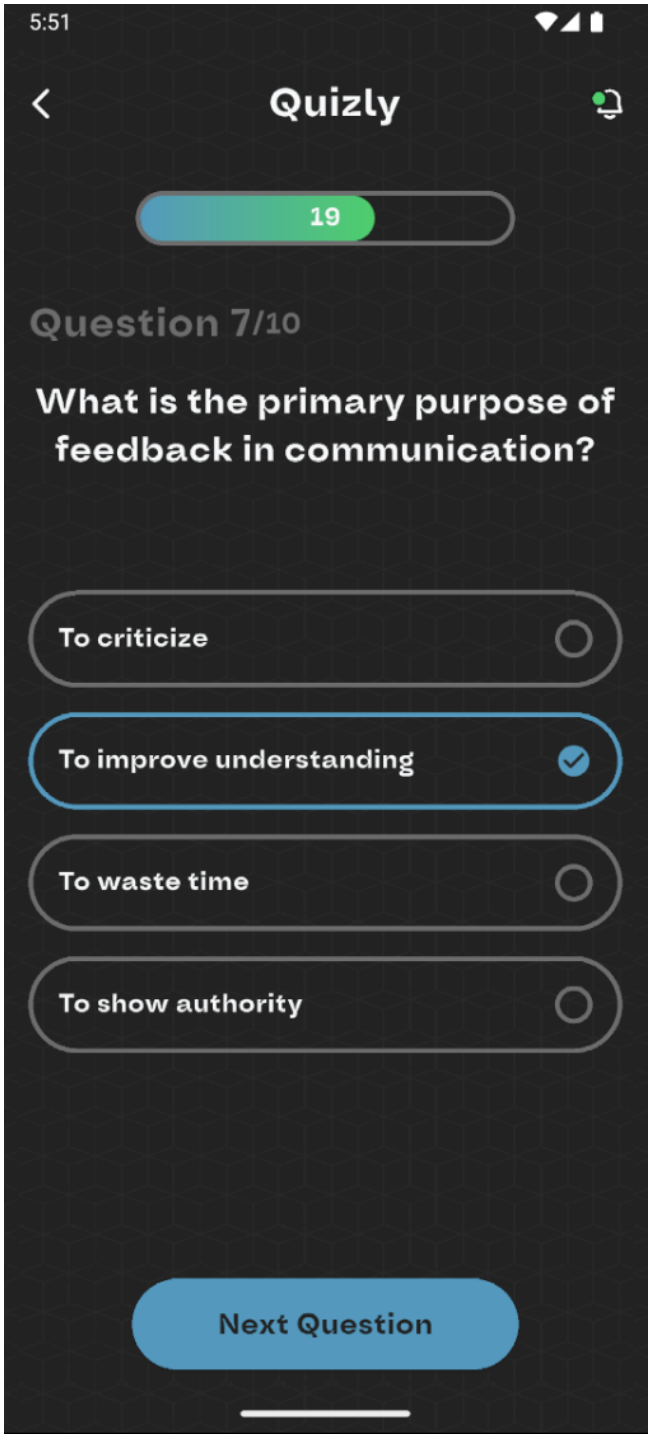


Figure 3.17: Quizly interface.

This figure 3.18 Represents the interface that shows the quizz's results :

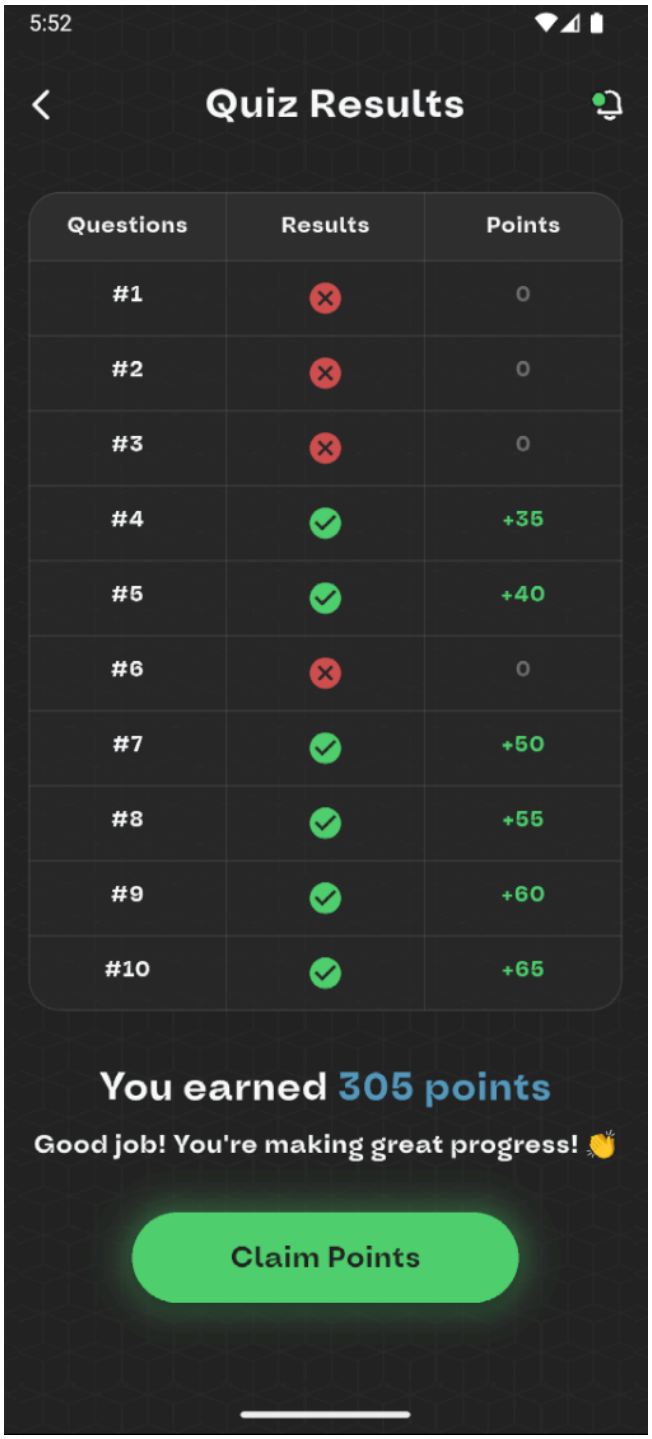


Figure 3.18: Quizz's result interface.

This figure 3.19 Represents the interface of the card information if the student wants to pay instead of gaining points to watch premium courses :

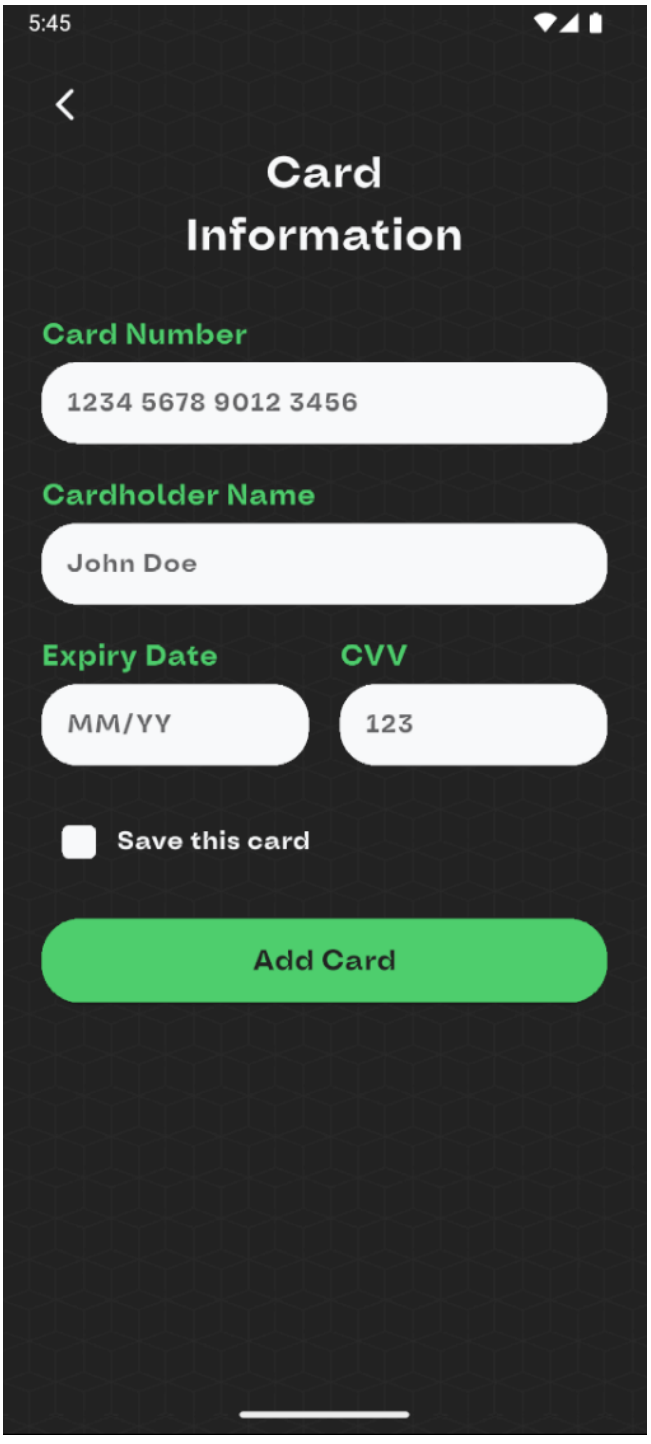


Figure 3.19: Card information interface.

3.4.4 The Sign up and the log in for the teacher and the student

This figure 3.20 Represents the sign up interface :

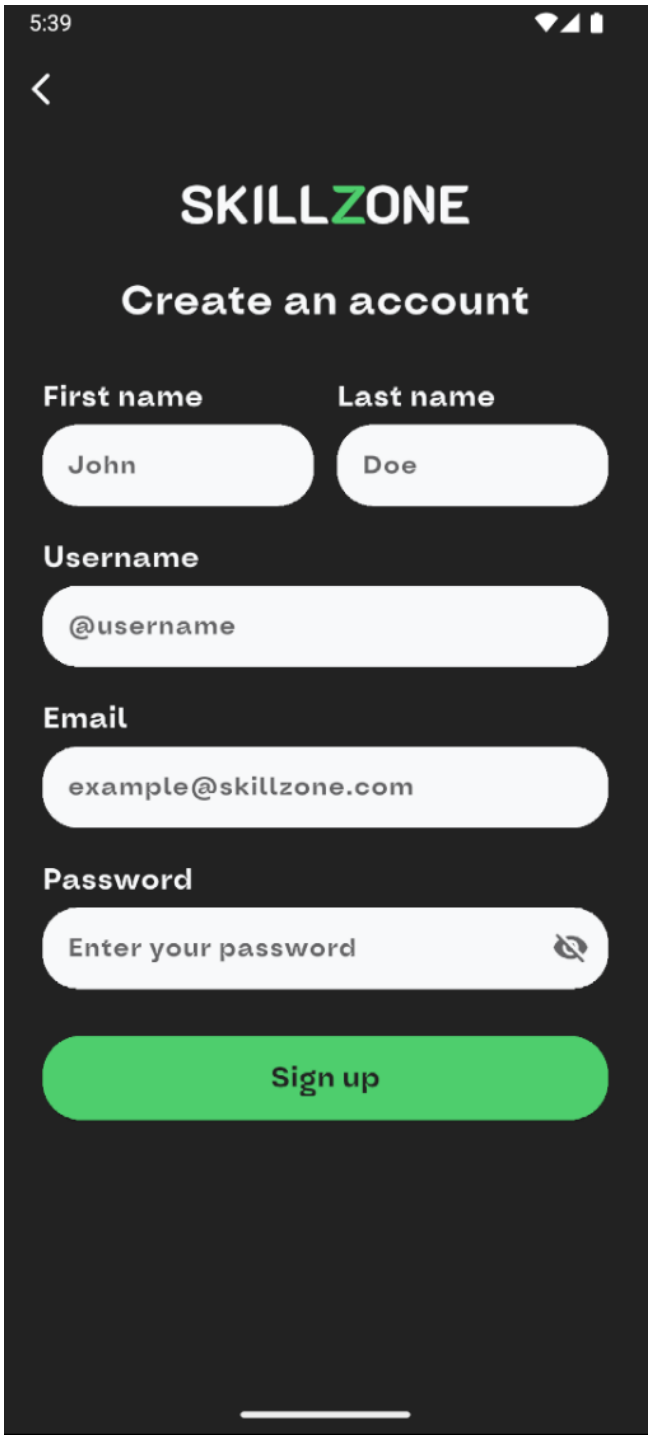


Figure 3.20: Sign up interface.

This figure 3.21 Represents the log in interface :

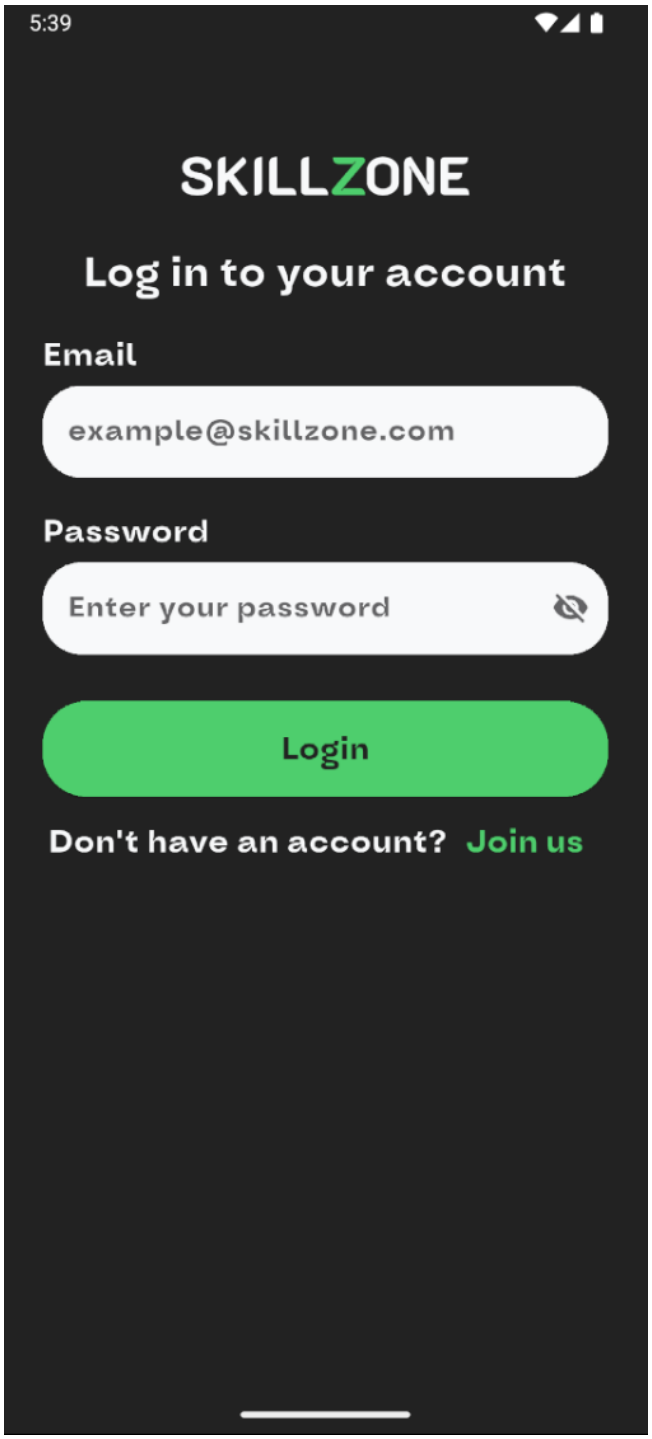


Figure 3.21: Log in interface.

This figure 3.23 Represents the interface in which the user choose if he is whether a student or a teacher :

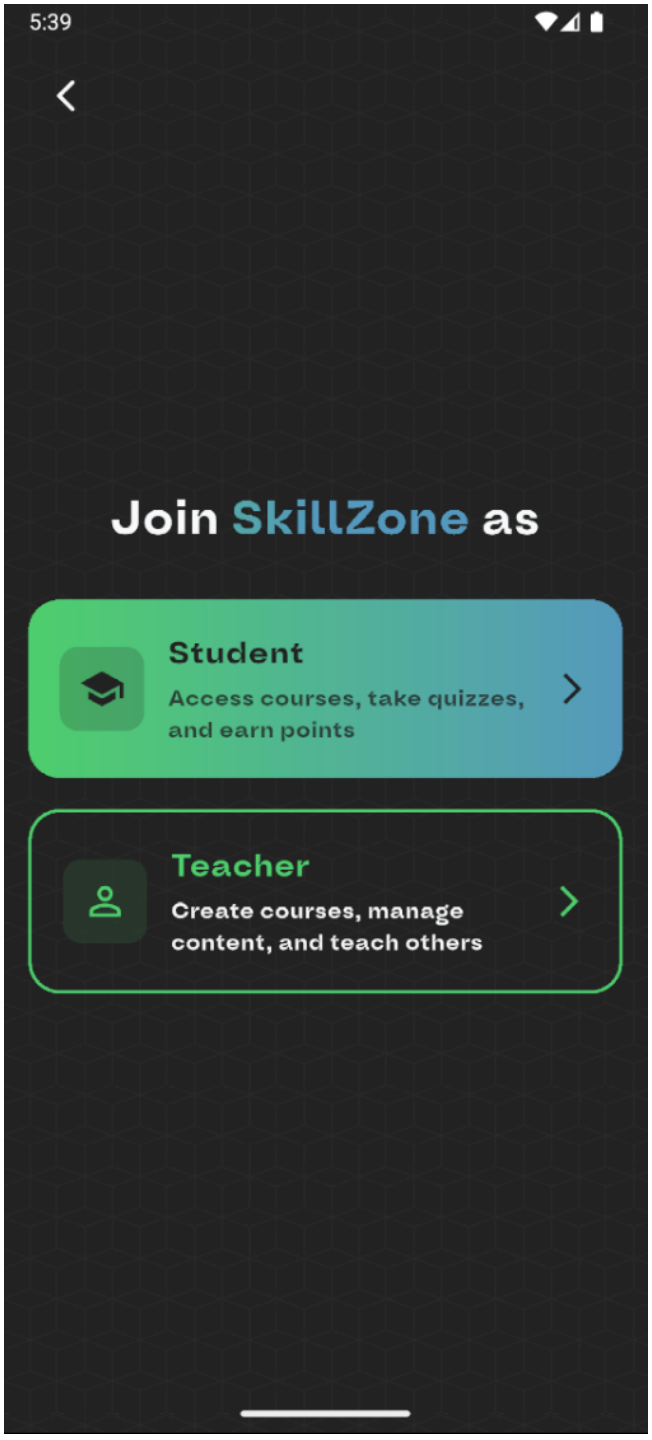


Figure 3.22: Account type interface.

This figure 3.23 Represents the interface in which the user Enters the verification code sent to their email :

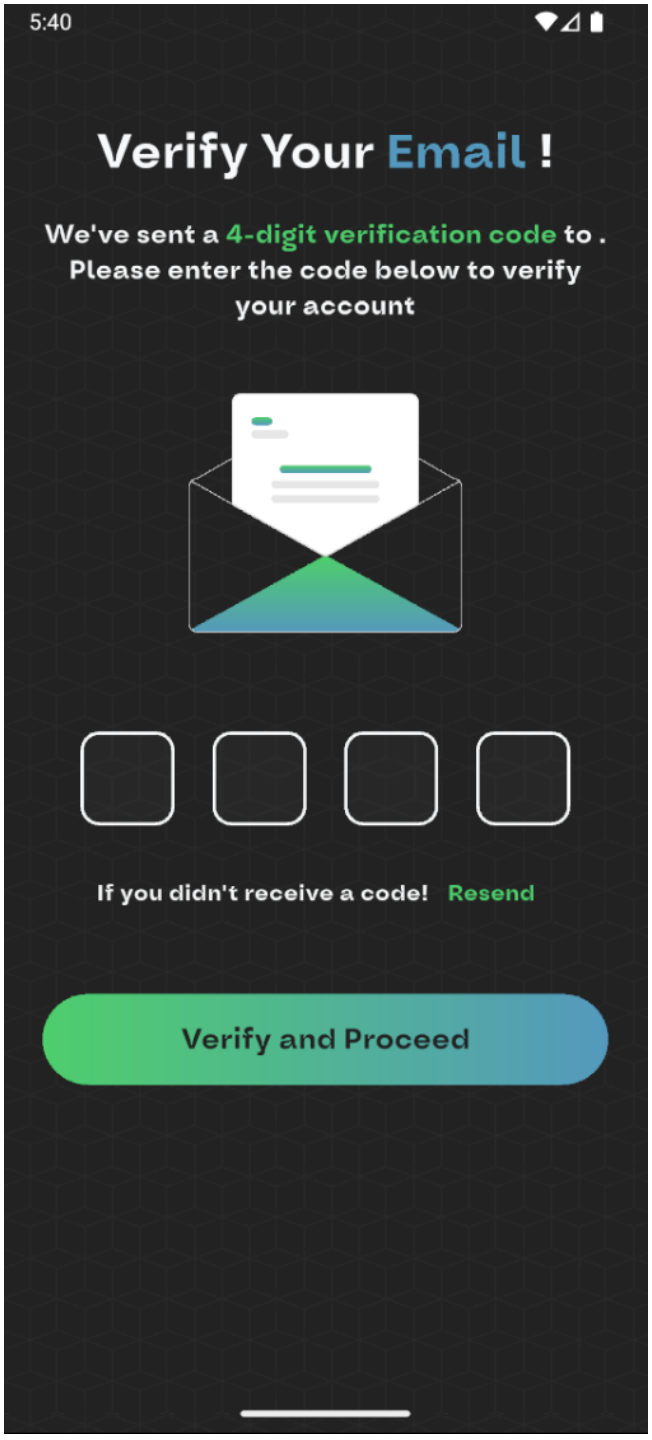


Figure 3.23: Account type interface.

3.4.5 The website's landing page

This figure 3.24 Represents the interface of Skillzone's website home page :

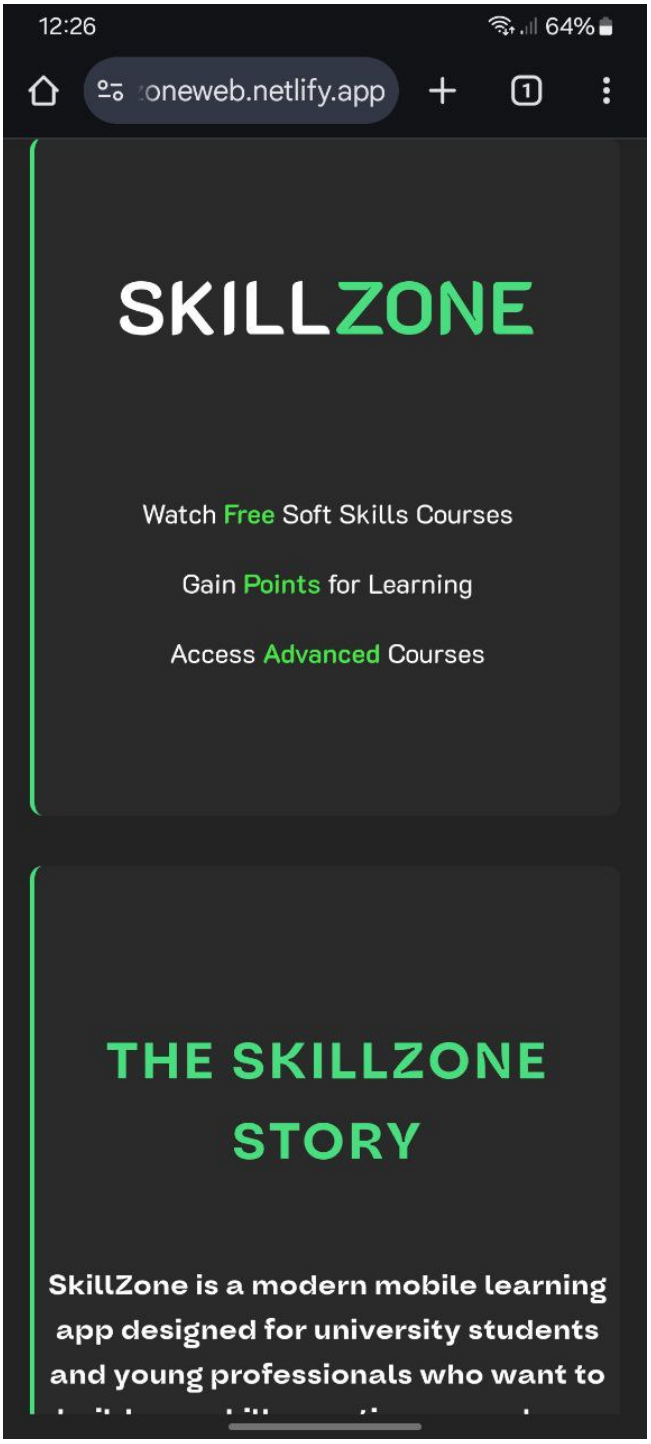


Figure 3.24: home page

3.4.6 The website's about us page

This figure 3.25 Represents the interface where the user can get all info about Skillzone its called about us page :

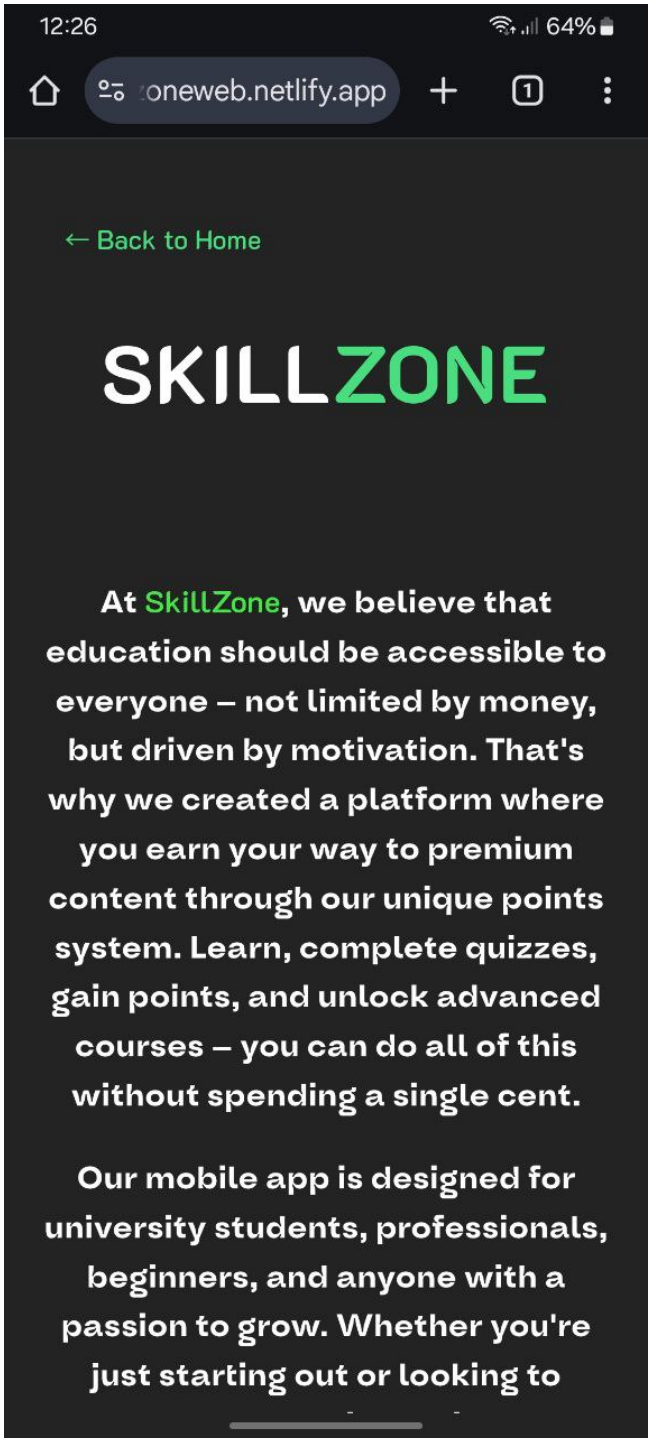


Figure 3.25: About us page.

3.4.7 The website's terms of service page

This figure 3.26 Represents the interface where the user can get all the conditions of the usage of Skillzone :

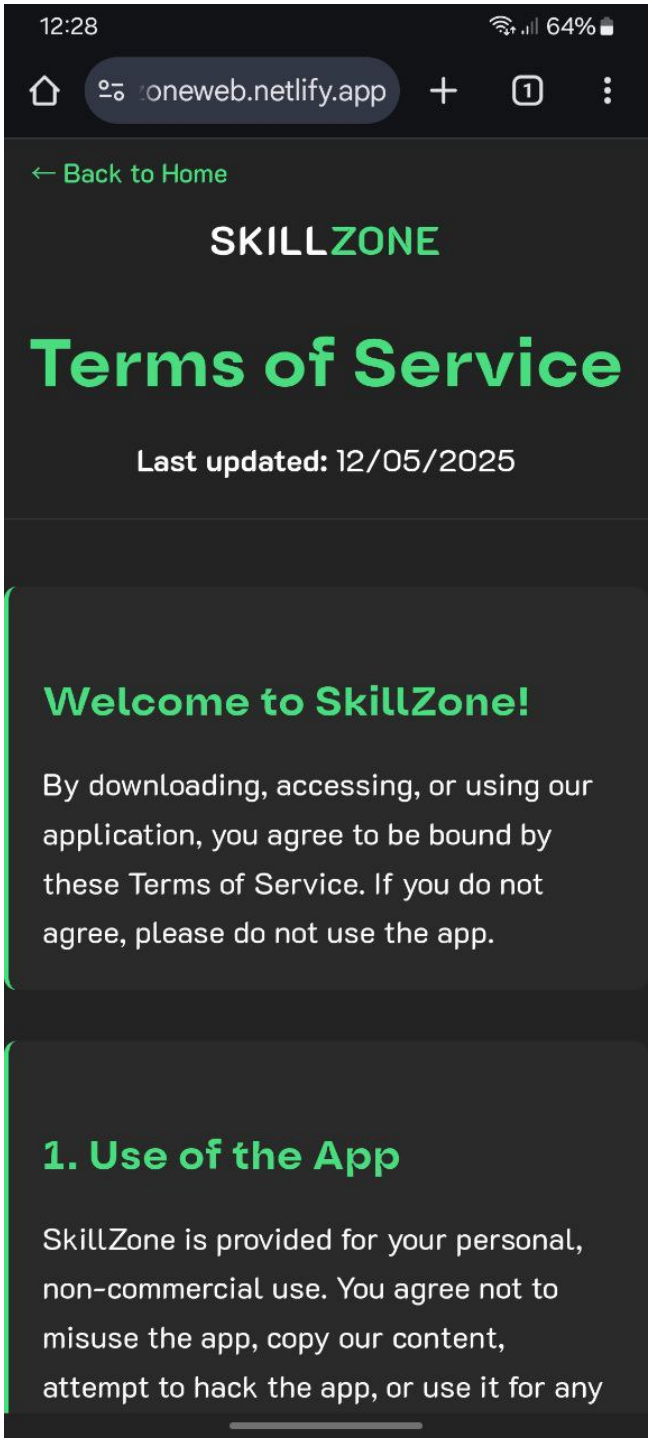


Figure 3.26: About us page.

3.5 Conclusion

This final section of our project was devoted to the implementation phase, which allowed us to achieve the objectives outlined in the specifications. We introduced the development environment and the main tools used to design our mobile application. We also presented the mobile application through a hierarchical view of its interfaces, detailing the key interfaces to clearly illustrate the work that has been accomplished.

General conclusion

In the scope of our project this year, we set our goal to design and implement a mobile application in the field of education and skill development titled SkillZone, aiming to support students in showcasing their skills and gaining recognition in a competitive environment. Our intention was to create a space where students can not only upload and organize their completed courses, but also track their progress and highlight their learning achievements in a structured, visual format. Through this platform, we aim to encourage self-learning, promote motivation, and facilitate access to relevant opportunities based on demonstrated competencies. To achieve this, our team began by conducting a preliminary study to analyze the needs of our users and review similar platforms. We then defined the functional and non-functional requirements of our system and proposed a solution tailored to the specific needs of Algerian students. Following this, we modeled our system using various diagrams to clarify the architecture and interactions within SkillZone. We chose technologies such as Flutter,react and Django for their suitability in building responsive and scalable applications. Finally, we designed and presented the main user interfaces of the SkillZone platform, offering a clear vision of how users will interact with and benefit from the system.

Bibliography

- Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2015.
- Pressman, Roger S. *Software Engineering: A Practitioner's Approach.* McGraw-Hill, 2010.
- Gamma, Erich, et al. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994.

Webography

- `https://codewithmosh.com/`
- `https://www.augmentcode.com/`
- `https://fr.overleaf.com`
- `https://www.w3schools.com/`
- `https://chat.openai.com/`
- `https://www.deepseek.com/`
- `https://netninja.dev/`
- `https://www.google.com/`
- `https://www.drawio.com/`
- `https://www.coursera.org/`
- `https://www.udemy.com/`
- `https://openclassrooms.com/`